# STUDY GUIDE FOR MODULE NO. 1

# INTRODUCTORY CONCEPTS TO PROGRAMMING

## MODULE OVERVIEW

This module aims to enable you to gain and demonstrate knowledge of computer systems, programming tools and basic steps in problem solving as preparation to understanding programming. The last section of this module provides you with an example on how to translate an algorithm design into C++ program codes. In addition, basic debugging technique is illustrated. Short discussions and self-tests were included to enrich learning of the student. Finally, an activity will have to be accomplished to assess your proficiency.

## MODULE LEARNING OBJECTIVES

By the end of this module, you should be able to:

- Demonstrate an understanding of how computer systems work through answering questions relevant to its components.
- Identify classifications and examples of programming languages.
- Differentiate compilers from linkers.
- Identify C++ programming tools.
- Design program solution on a given problem using pseudocode or flowchart.
- Translate an algorithm design into C++ program codes.

## LEARNING CONTENTS (Computer Systems)

### Introduction

Computer systems consist of hardware and software components. The computer hardware components are physical parts found within computer case while some parts may be connected externally. The computer system requires direction to perform various operations. Thus, the computer system needs a program that directs the computer hardware. Commonly, the computer program is referred to as software.

### Unit learning outcomes

By the end of this unit you should be able to:

- Identify components of computer systems.
- Distinguish types and examples of programming languages.
- Differentiate compilers from linkers.

### 1.1 Computer Systems

- A computer program is a set of instructions for a computer to follow. **Computer software** is the collection of programs used by a computer including:
  - Editors
  - Translators
  - System Managers
- Hardware pertains to the physical parts of the computer system. Three main classes of computers include:
  - PCs (Personal Computer)
    - Relatively small used by one person at a time
  - Workstation
    - Larger and more powerful than a PC
  - Mainframe

- Still larger
- Requires support staff
- Shared by multiple users

- Computer organization has five main components namely:
  - 1. Input Devices
    - Allows communication to the computer
  - 2. Output Devices
    - Allows communication to the user
  - 3. Processor (CPU)
  - 4. Main memory
    - Memory locations containing the running program
  - 5. Secondary memory
    - Permanent record of data often on a disk
    - Main memory
    - Long list of memory locations
    - Each contains zeros and ones
    - Can change during program execution
    - Binary Digit or Bit
    - A digit can only be zero or one
    - Byte
    - Each memory location has eight bits
    - Address
    - Number that identifies a memory location
- Secondary memory
  - Stores instructions and data between sessions
  - A file stores data or instructions in secondary memory
- Memory Access
  - Random Access
    - Usually called RAM
      - Computer can directly access any memory location
  - Sequential Access
    - Data is generally found by searching through other items first
    - More common in secondary memory
- The Processor
  - Typically called the CPU
    - Central Processing Unit
  - Follows program instructions
  - Typical capabilities of CPU include:
    - Add
    - Subtract
    - Multiply
    - Divide
    - Move data from location to location
- The Operating System
  - The operating system
    - Allows us to communicate with the computer
  - Is a program
  - Allocates the computer's resources
  - Responds to user requests to run other programs
  - Common operating systems include:
  - UNIX DOS Macintosh
  - Linux Windows VMS
- High level languages
  - Common programming languages include…
    - C C++ Java Pascal Visual Basic
    - FORTRAN COBOL Lisp Scheme Ada

- These high-level languages
- Resemble human languages
- Are designed to be easy to read and write
- Use more complicated instructions than the CPU can follow
- Must be translated to zeros and ones for the CPU to execute a program

❖ Low level languages

- An assembly language command such as
  - ADD X Y Z
  - Might mean add the values found at x and y in memory, and store the result in location z.
  - Assembly language must be translated to machine language (zeros and ones)
    - 0110 1001 1010 1011
  - The CPU can follow machine language

❖ Programming languages

- Machine Language (first generation)
  - The computer's 'native language'
  - Composed of binary digits(0s, 1s)
  - E.g. 0110 1001 1010 1011
  - The only language that computers understand
- Assembly Language (second generation)
  - One-to-one correspondence to machine language (mnemonic rather that binary digits)
  - E.g. ADD X Y Z
  - Assembler – program that translates an assembly language program into machine language
- Procedural Languages (third generation)
  - One instruction translates into many machine language instructions
  - Programs describe the computer's processing step-by-step
  - Closer to natural language; uses common words rather than abbreviated mnemonics
  - Examples: C, C++, Java, Fortran, QuickBasic
  - Compiler – translates the entire program at once
  - Interpreter – translates and executes one source program statement at a time
- Non-procedural Languages (fourth generation)
  - Allows the user to specify the desired result without having to specify the detailed procedures needed for achieving the result
  - Example: database query language – SQL
  - Can be used by non-technical users
- Natural Language Programming Languages (fifth generation (intelligent) languages)
  - Translates natural languages into a structured, machine-readable form
  - Are extremely complex and experimental

❖ Current Programming Languages

- Object-Oriented Programming Languages (OOP)
  - Based on objects – packaging data and the instructions about what to do with that data together
- Examples: Java, C++
- Visual Programming Languages
  - Used with a graphical environment
  - Examples: Visual Basic and Visual C++
  - Popular to non-technical users
- Hypertext Markup Language (HTML)

        - Standard language used in World Wide Web
        - Contains text, images, and other types of information such as data files, audio, video and executable computer programs
    - Extensible Markup Language (XML)
        - Improved on Web document functionality
    - Virtual Reality Modeling Language (VRML)
        - A file format for describing three-dimensional (3D) interactive worlds and objects
        - Can be used with the World Wide Web
- Compilers
    - Translate high-level language to machine language
        - Source code
            - The original program in a high-level language
        - Object Code
            - The translated version in machine language
- Linkers
    - Some programs we use are already compiled
        - Their object code is available for us to use
        - For example: Input and output routines
    - A Linker combines
        - The object code for the programs we write and
        - The object code for the pre-compiled routines into
        - The machine language program the CPU can run

You can find further readings on computer systems at https://www.pearsonhighered.com/assets/samplechapter/0/3/2/1/0321537114.pdf (Chapter 1), programming concepts at https://beginnersbook.com/2017/08/c-plus-plus-tutorial-for-beginners/ and https://www.tutorialspoint.com/cplusplus/index.htm

## LEARNING ACTIVITY 1

Directions: Test your present knowledge about computer systems by selecting the option that corresponds to the correct word or phrase that will complete the statement. Encircle the letter of the correct answer.

1. The collection of programs used by a computer is referred to as _____.
   **A.** Hardware **B.** Software **C.** Processor **D.** Main memory

2. Which of the following is an example of computer hardware?
   **A.** Personal Computer **B.** Editors **C.** Translators **D.** System Managers

3. Which of the following contains the running program?
   **A.** Processor **B.** Secondary memory **C.** Main memory **D.** Input

4. This is a volatile type of memory that is used only for temporary storage while a program is running.
   **A.** RAM **B.** secondary storage **C.** the disk drive **D.** the USB drive

5. A component that collects data from people or other devices and sends it to the computer is called _____.
   **A.** an output device **B.** an input device **C.** a secondary storage device **D.** memory

6. Programming languages that resemble human languages are classified as _____ languages.
   **A.** high level **B.** low level **C.** operating system **D.** processor

7. Languages closer to natural language that uses common words rather than abbreviated mnemonics are termed as _____.
**A.** assembly language **B.** machine language **C.** assembler **D.** procedural languages
8. The one that translates and executes one source program statement at a time is _____.
**A.** interpreter **B.** compiler **C.** linker **D.** object code
9. The one that combines the object code for the programs we write and the object code for the pre-compiled routines into the machine language program the CPU can run is the _____.
**A.** linker **B.** compiler **C.** source code **D.** object code
10. The one that translates high level language to machine language are _____.
**A.** linkers **B.** compilers **C.** source code **D.** object code

**Activity:**
Directions: Complete this activity by answering the questions in your own words shortly but concisely on the spaces provided.

1. What is a program?

2. What is the difference between procedural and non-procedural programming languages?

3. Why is the CPU the most important component in a computer?

4. What are the functions of input and output devices in a computer system?

5. What is the difference between a compiler and an interpreter?

## LEARNING CONTENTS (C++ Programming Tools)

### Introduction

- The very first thing you need to do programming is a compiler. A compiler reads the entire program and converts it into object code, which is a translation of the program source code into a form that the computer can execute directly. There are a number of commercial Integrated Development Environments (IDEs) that support C/C++. There are also some good free IDEs available, including Bloodshed Dev-C++ and Code::Blocks. These are completely self-contained environment for creating, compiling, linking and testing C++ programs. It incorporates a range of fully integrated tools designed to make the whole process of writing C++ programs easy. In this unit, you will get a view of how to install the IDE and how the IDE works (Dev-C++ and Code::Blocks, in particular).

### Unit learning outcomes

By the end of this unit you should be able to:

- Identify the main areas in the workspace of Integrated Development Environments (IDEs) for C++.
- Demonstrate understanding of the installation, setting up of IDEs and use of the fundamental parts such as the editor, compiler, linker and the libraries.

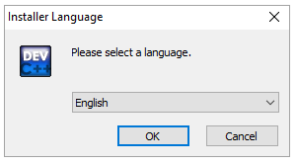
## 1.2 C++ Programming Tools

- C++ IDE (Integrated Development Environment) is a completely self-contained environment for creating, compiling, linking and testing C++ programs. It incorporates a range of fully integrated tools designed to make the whole process of writing C++ programs easy.
- You can begin by downloading the installer from these URL sites. These are free shareware software.
  - http://sourceforge.net/projects/orwelldevcpp/
  - http://sourceforge.net/projects/codeblocks/?source=dlp

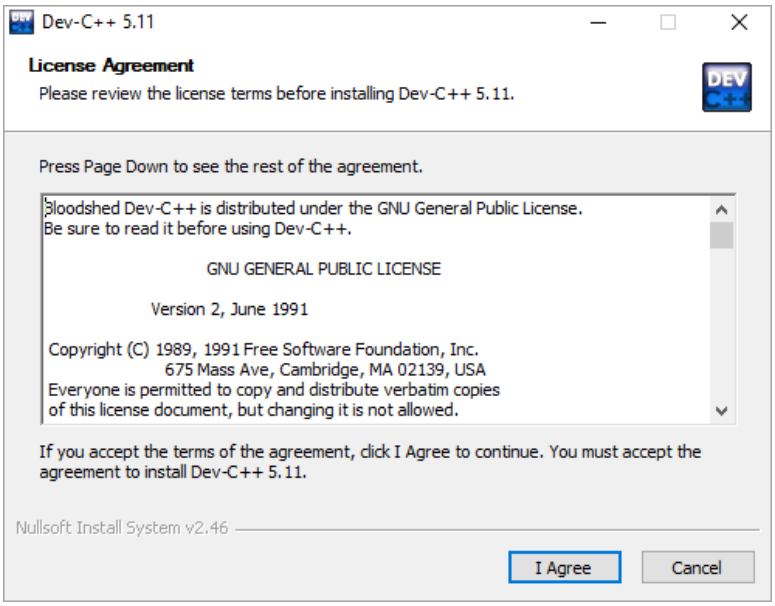
- Download both of them in your computer and learn them. Then choose the one which is most appealing and convenient for you.

### 1.2.1Installing Dev-C++

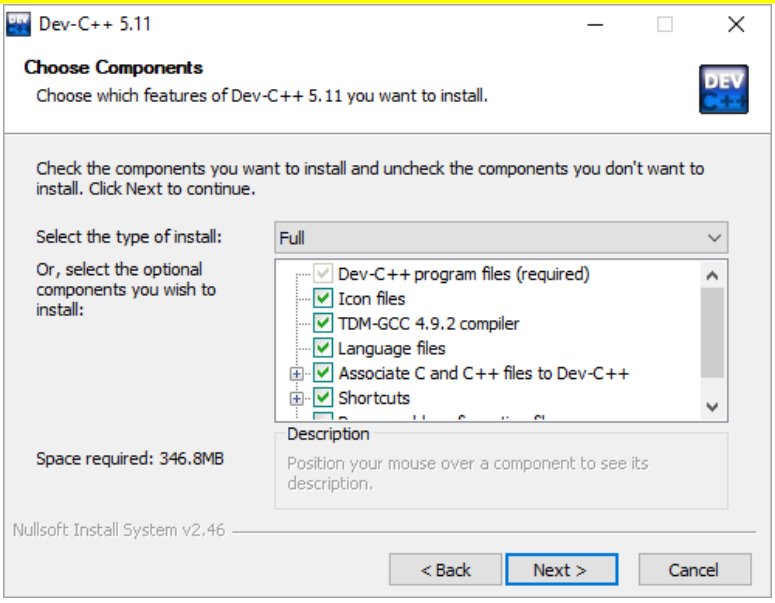
**#1)** The first step while we start the installer is to select the language of our choice as shown in the below screenshot.




**#2)** Once you select the appropriate language, you have to agree to the license agreement that pop-ups next.

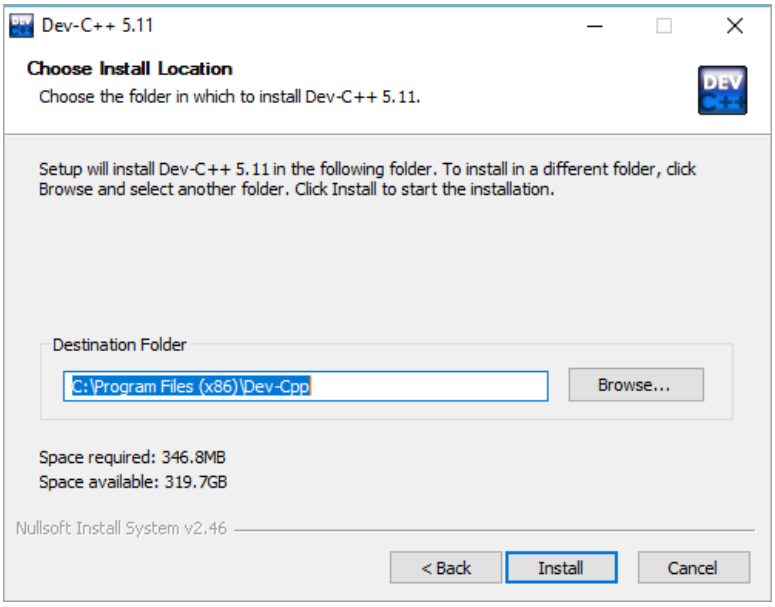



**#3)** Next, we are asked to select the components that we need to install as a part of the dev-C++ installation.



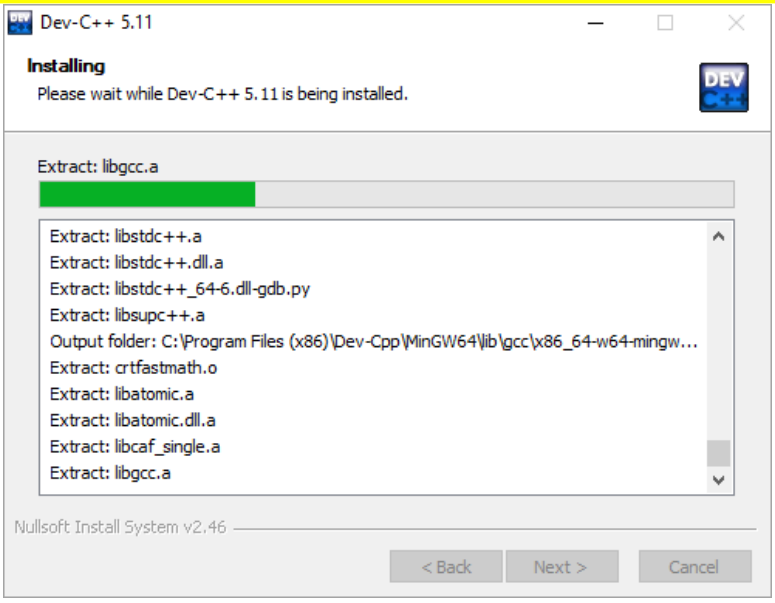

As shown in the above screenshot, we are provided with a list of components available for installation and a checkbox against each component. We can check/uncheck each box to indicate which components to install. Click next once the components are selected.

**#4)** Now the installer prompts the user for the destination folder where the dev-C++ files/libraries etc. are to be copied.




Once we provide the destination folder path, click on Install.

**#5)** The following screenshot shows the progress of the installation.




Once the installation is over, a “finish” dialog that signals the end of the installation appears. We click finish and then we can launch the dev-C++ IDE.

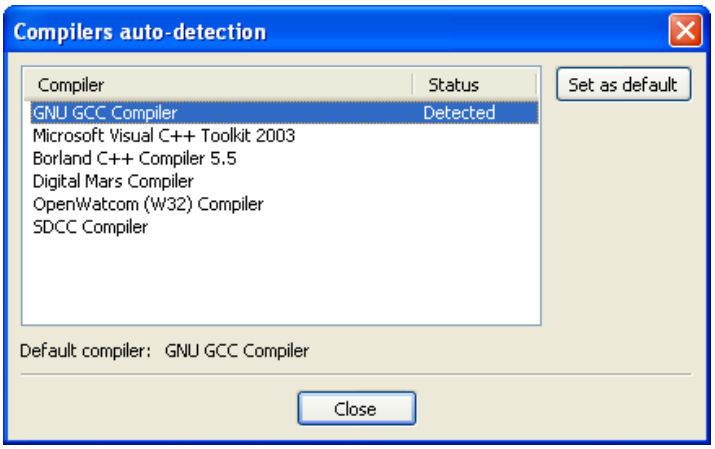
### 1.2.2 Installing C++ Code::Blocks

**#1)** Double-click the installer once it has completed downloading.

**#2)** Hit next several times. Other setup tutorials will assume you have installed in C:\Program Files\CodeBlocks (the default install location), but you may install elsewhere if you like. Do a Full Installation. Then, launch Code::Blocks.

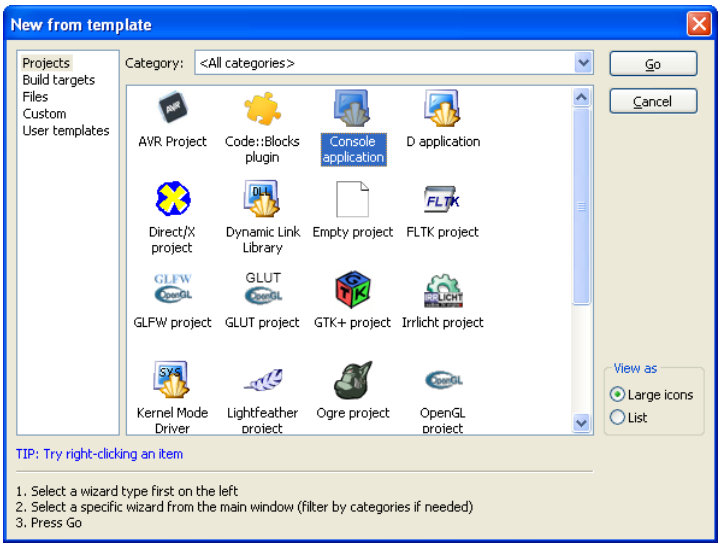
**#3)** Running in Code::Blocks

You will be prompted with a Compilers auto-detection window:



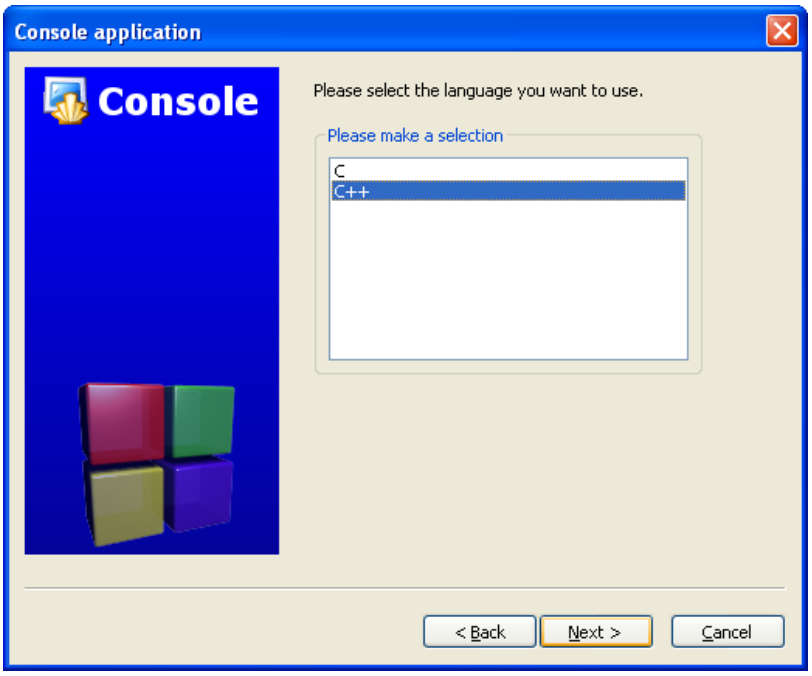

When you get the compiler auto-detection window, just hit OK. Code::Blocks may ask if you want to associate it as the default viewer for C/C++ files--I'd suggest you do. Click on the File menu, and under "New", select "Project..."

The following window will come up:



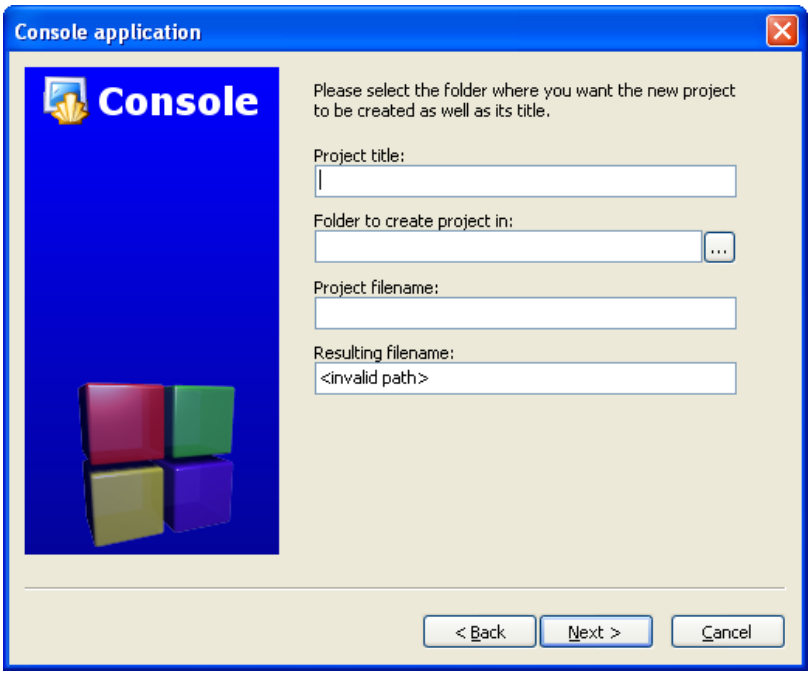

Click on "Console Application" and hit the "Go" button.

Click next until you get to the Language Selection Dialog:



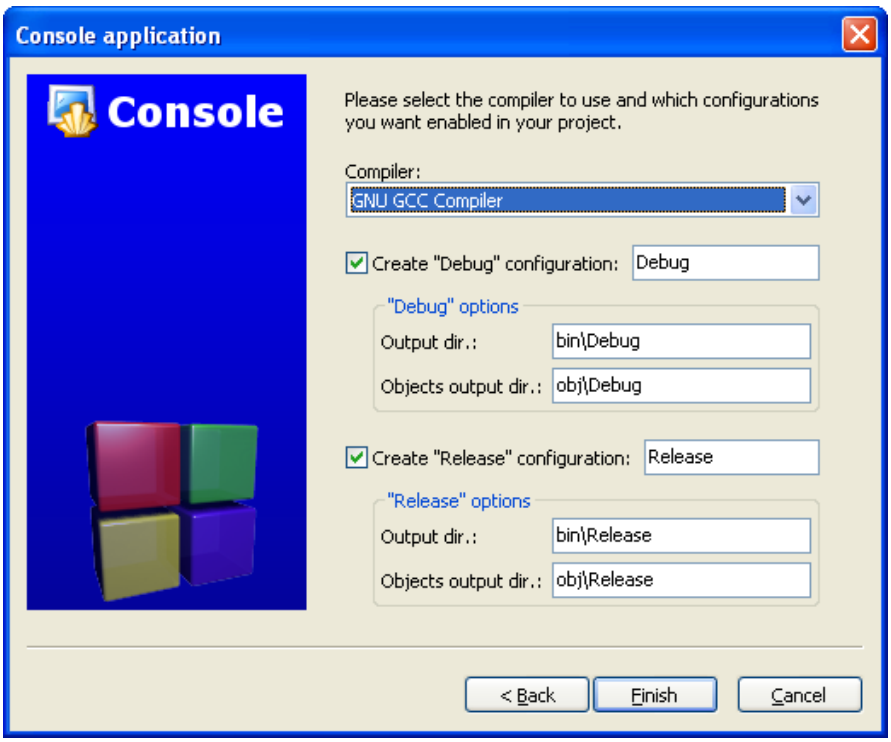

You'll be asked to choose whether you want to use C or C++. If you're not sure, use C++. Otherwise, choose based on the language you are learning. (You can find tutorials here on both C and C++.)

After clicking "Next", Code::Blocks will then prompt you with where you'd like to save the console application:



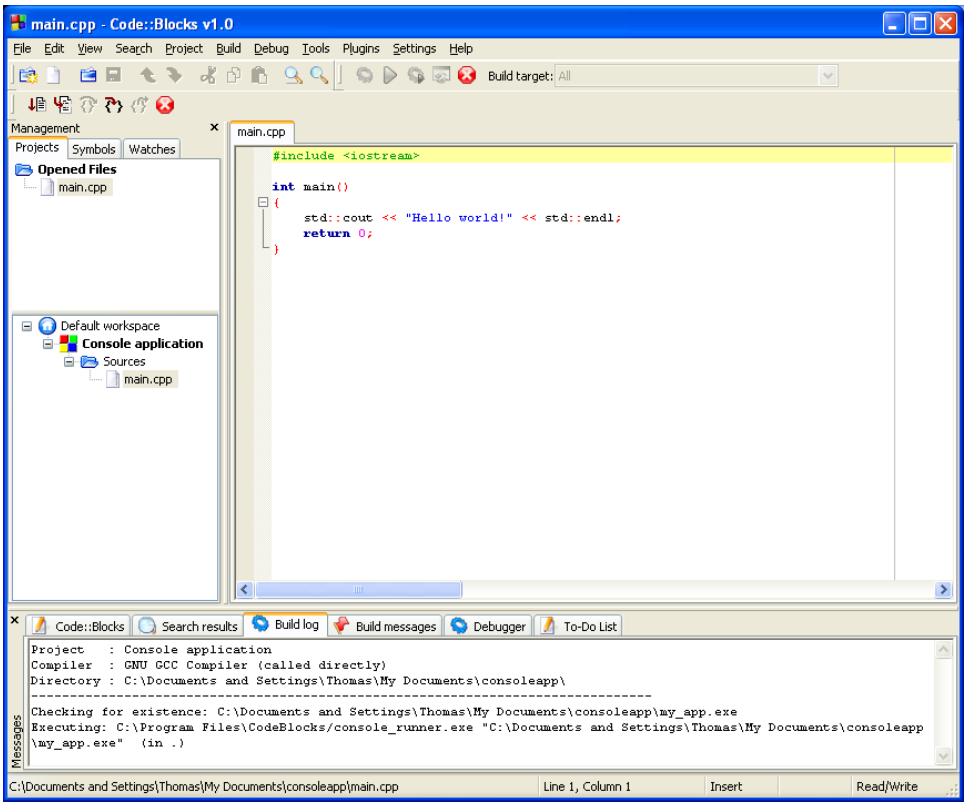

I'd recommend you put it in its own folder, as it may create several files (this is especially true if you create other types of projects). You will need to give your project a name, anything will be fine.

Clicking "Next" again will prompt you to set up your compiler:



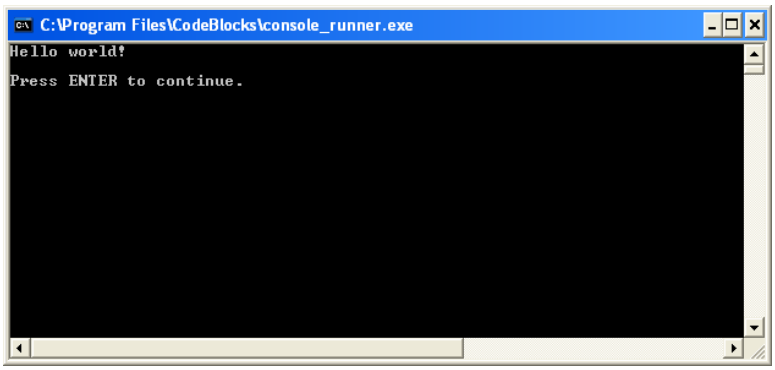

You don't need to do anything here. Just accept the defaults by hitting "Finish".

You can now open the main.cpp file on the left:




(You may need to expand the contents of the "Sources" folder if you don't see main.cpp.)

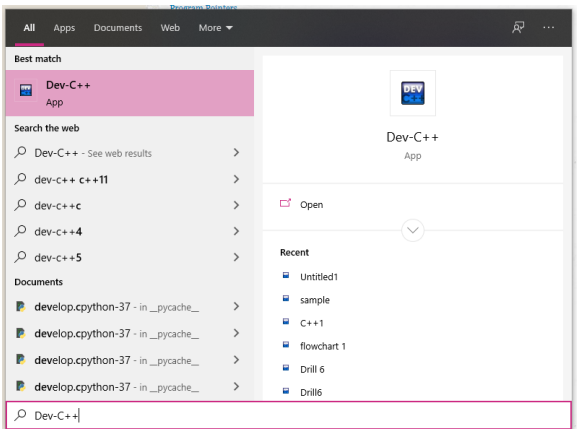
At this point, you will have your main.cpp file, which you can modify if you like. For now, it just says "Hello World!", so we can run it as is. Hit F9, which will first compile it and then run it.



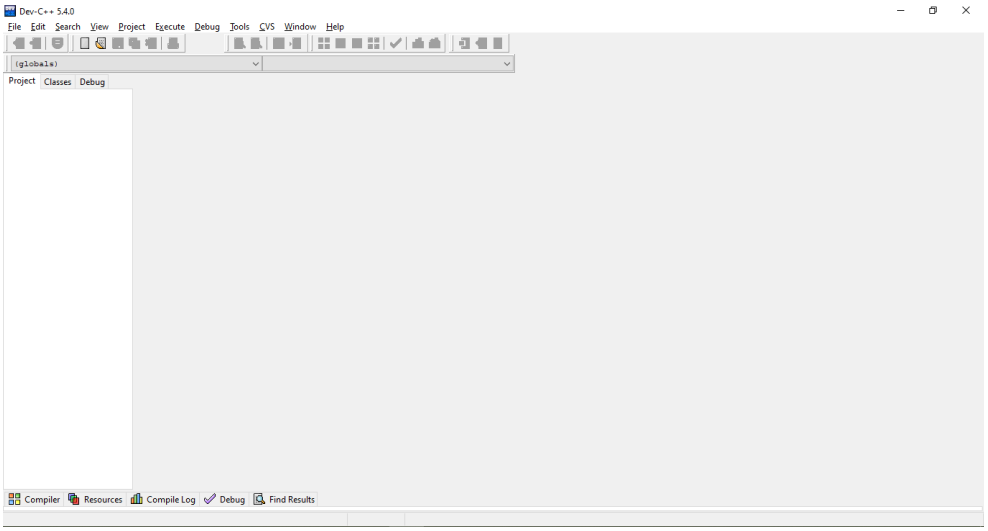

You now have a running program! You can simply edit main.cpp and then hit F9 to compile it and run it again.

### 1.2.3 Using Dev-C++

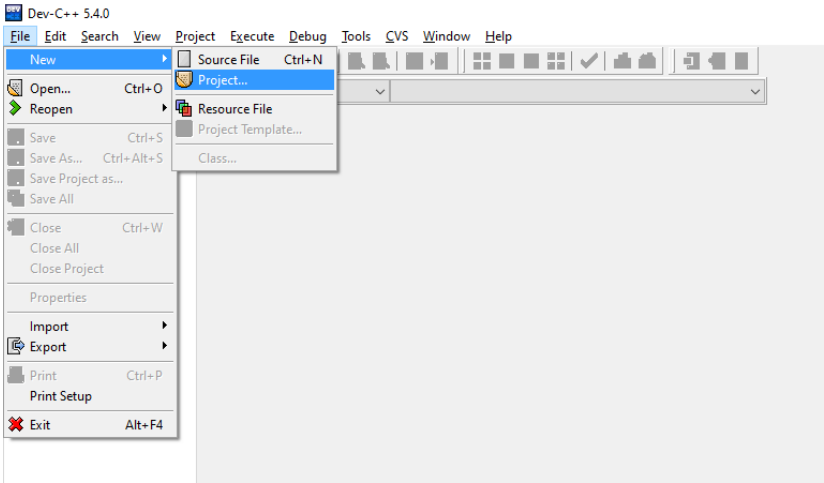
**#1)** Open the Dev-C++ IDE



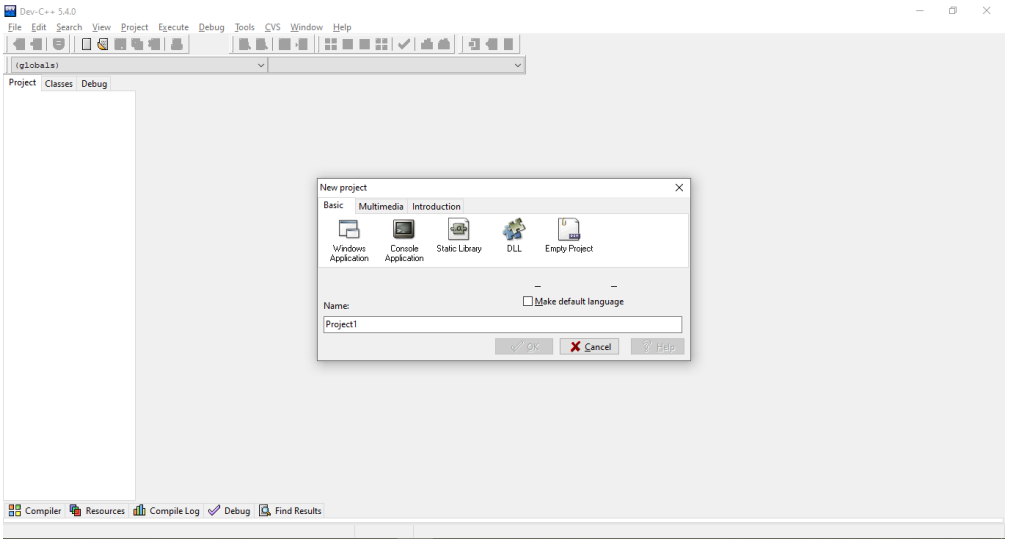

**#2)** Display Dev-C++ IDE

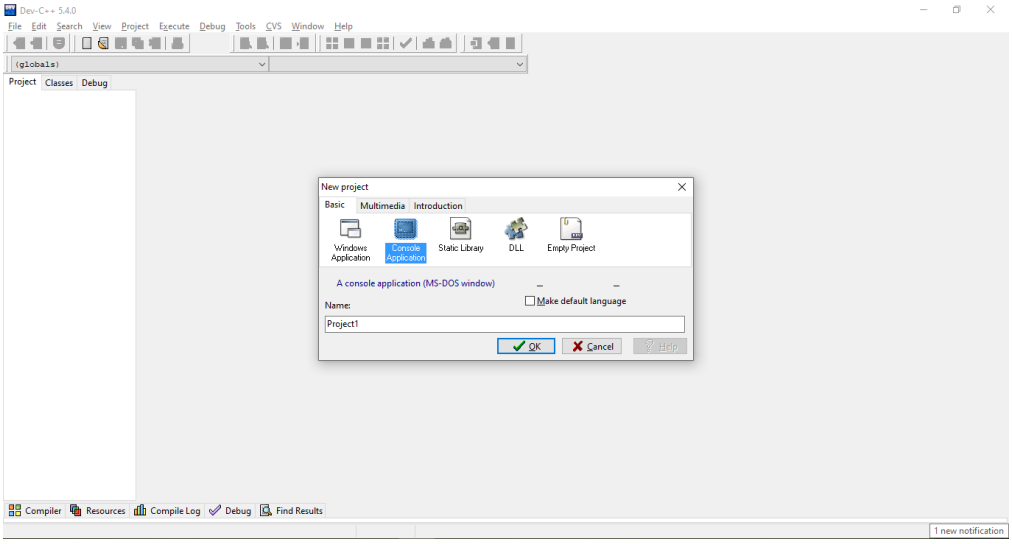



**#3)** Click Main Menu, New > Project

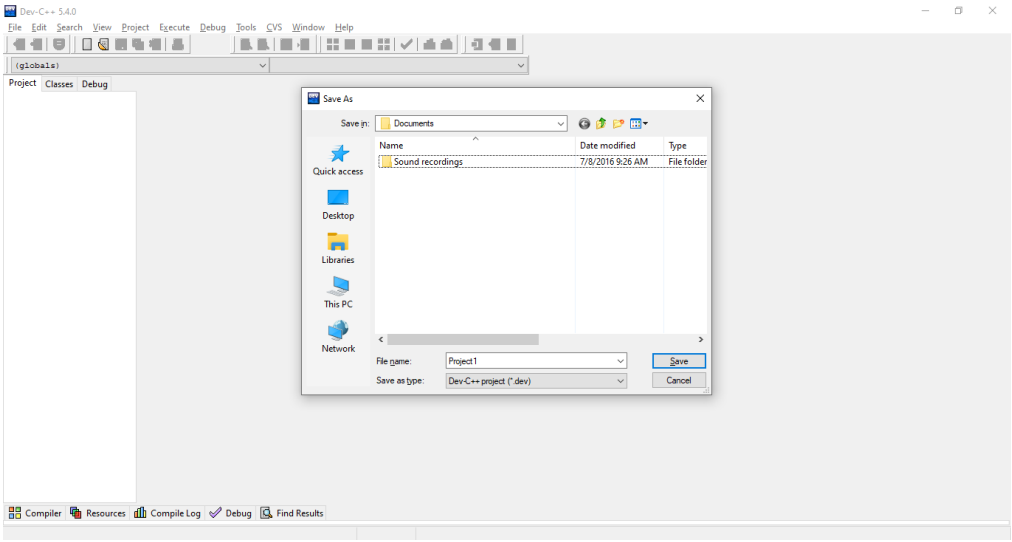


**#4)** Select "Console Application" & C++ Project
Enter a Project Name, Then Click "OK"

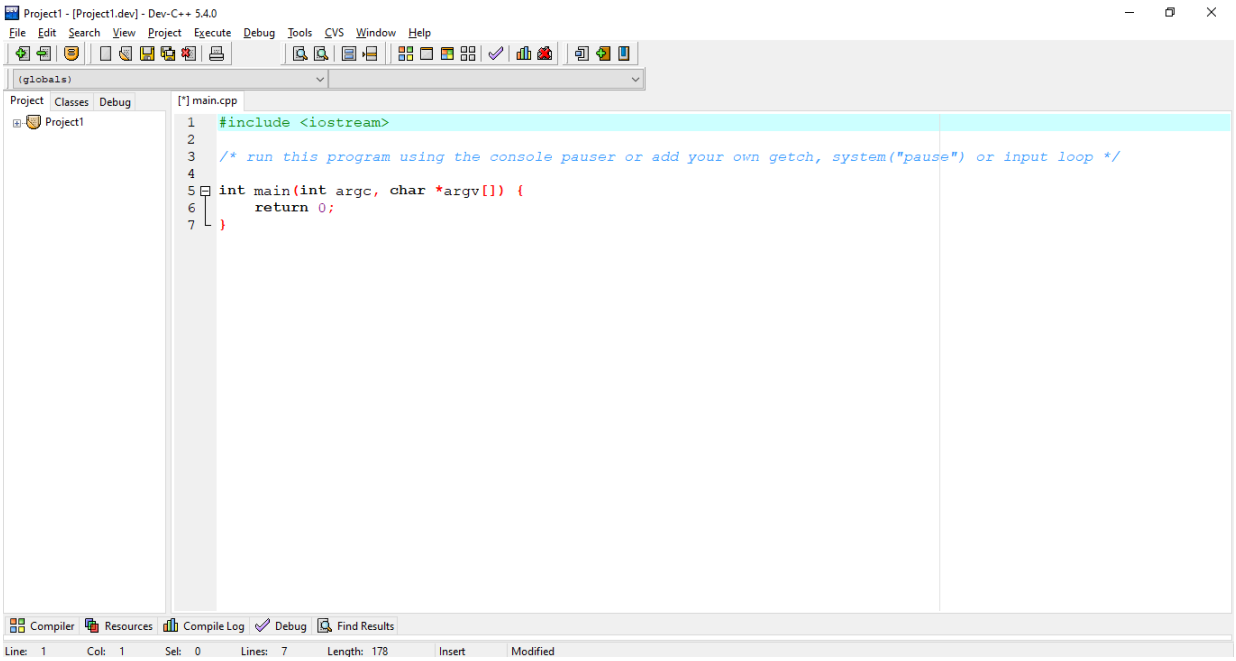


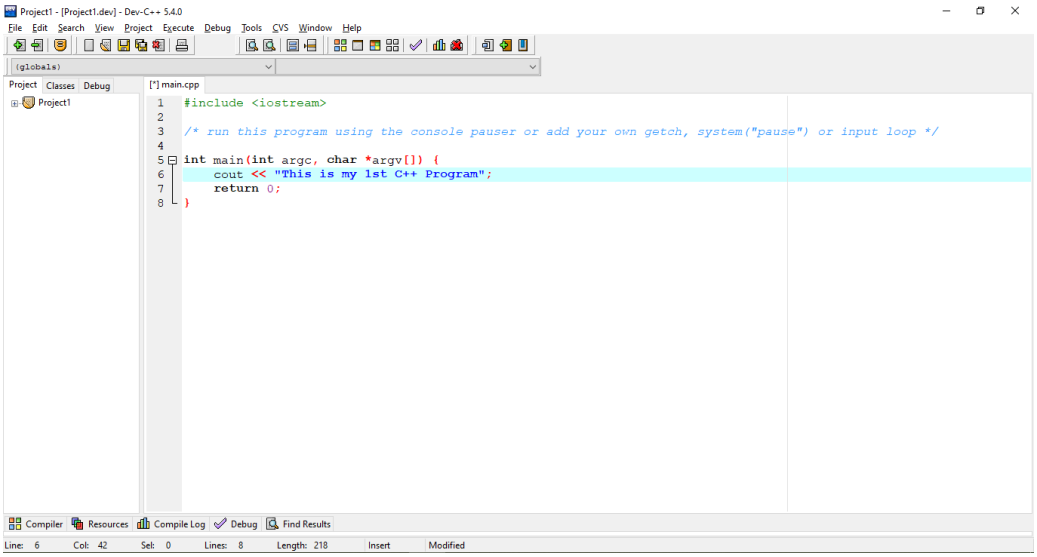

**#5) Click** "save" - to save Project



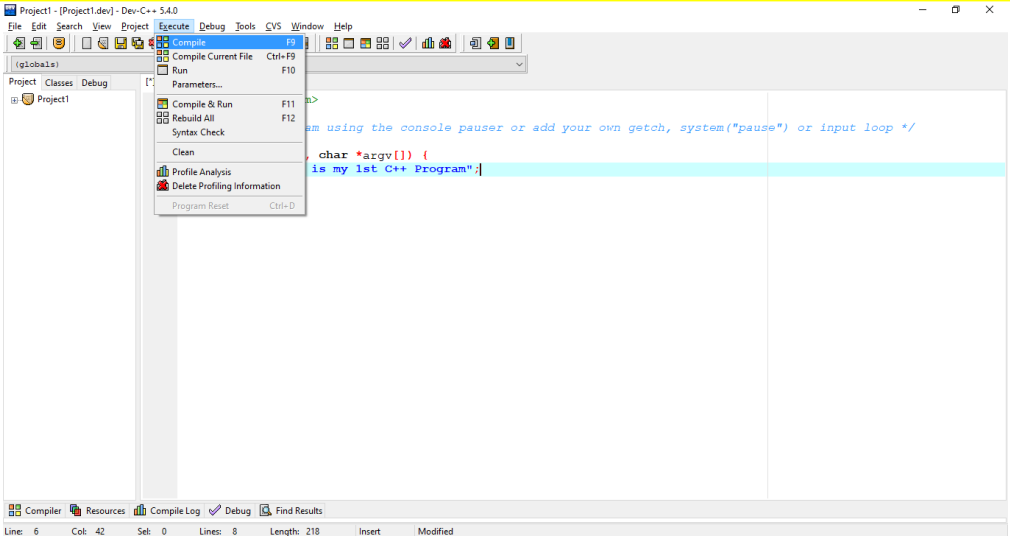
**#6)** Display the FileName with default Contents
or default Main Program Structure.



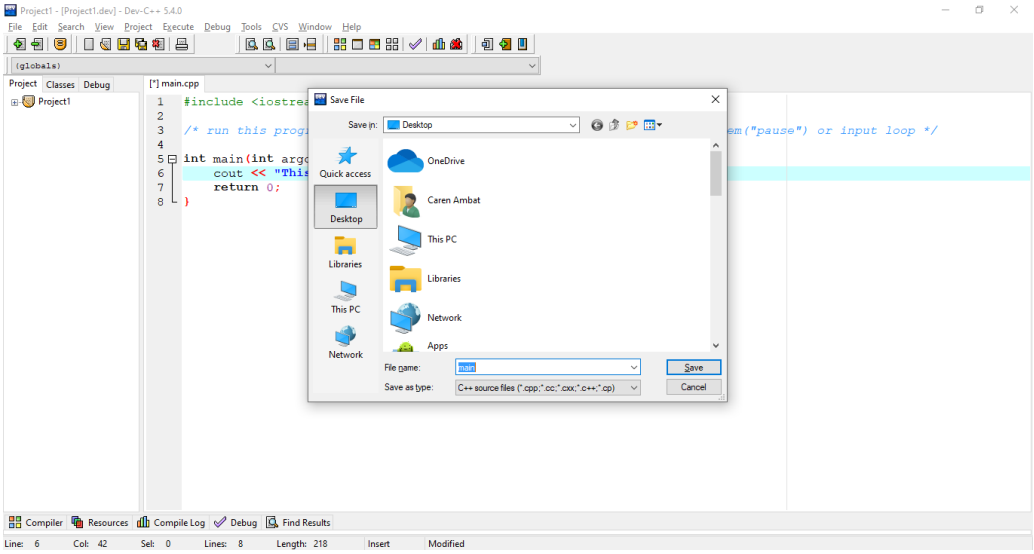

**#7)** Insert a statement
cout << "This is my 1st C++ Program";
to make program more meaningful.



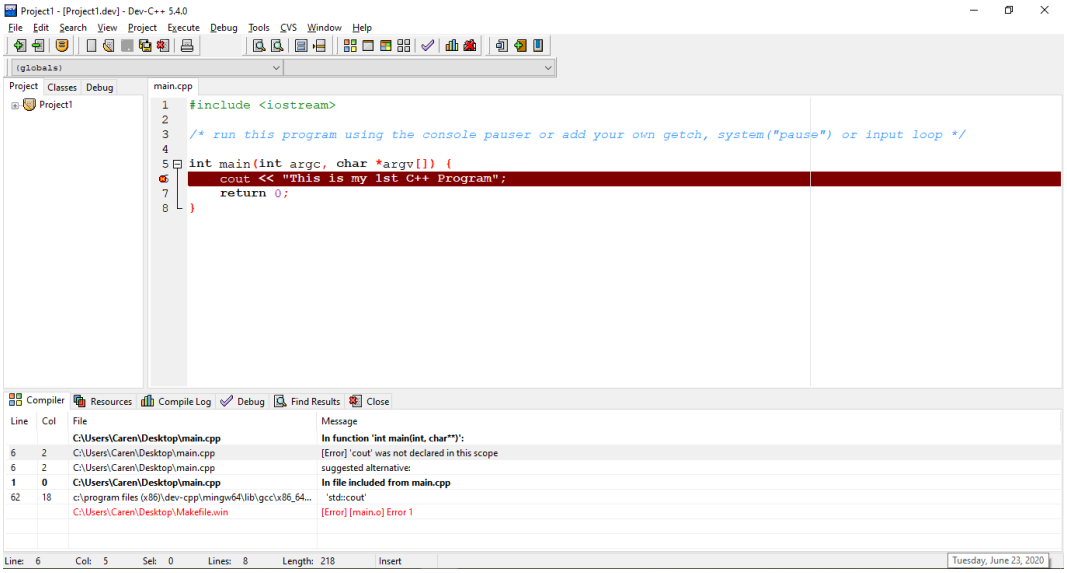

**#8)** Click Main Menu, Execute > Compile.



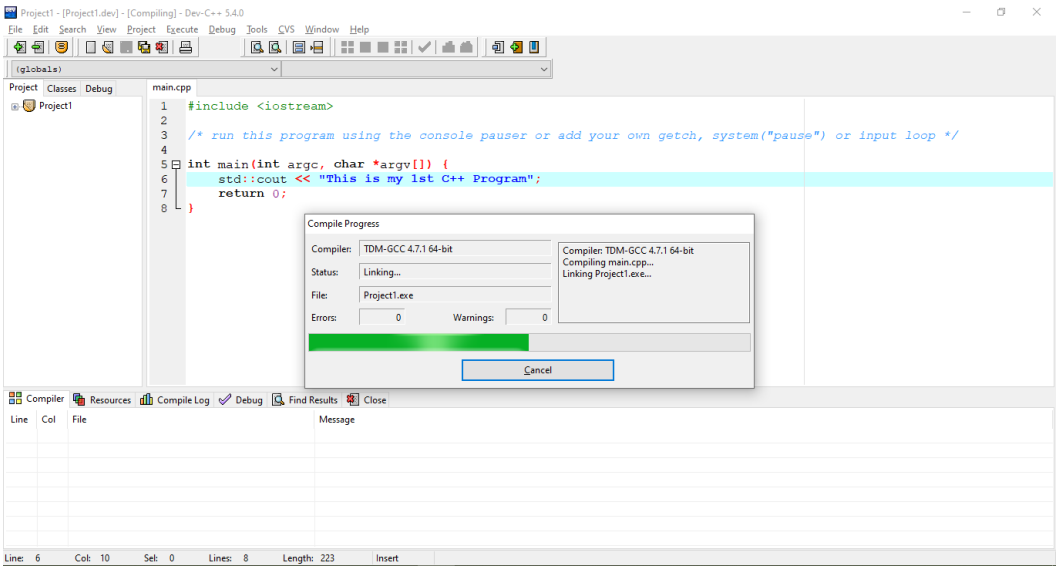

**#9)** *Type in a Name for Source Program*
*Click "Save"*



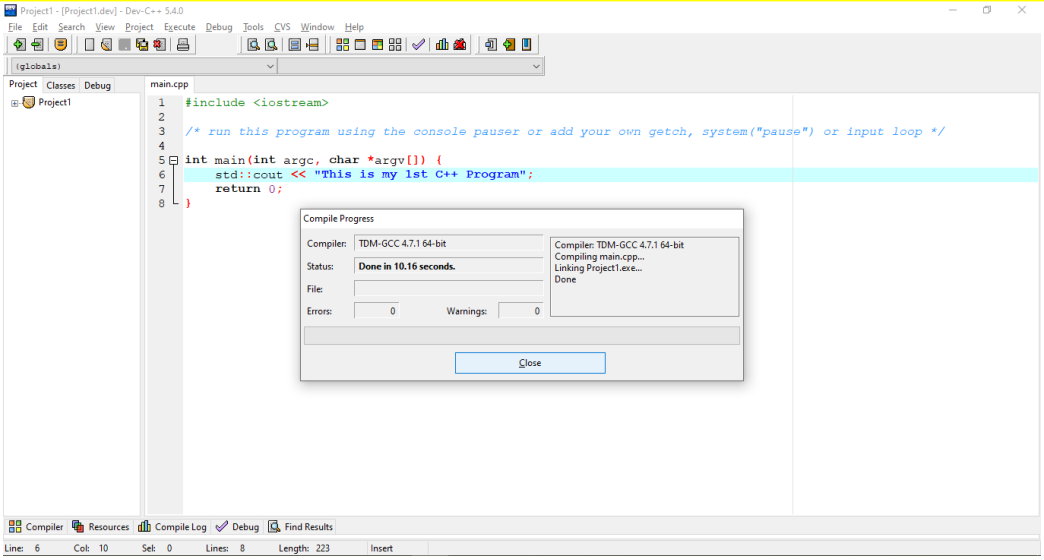

**#10)** Check for any Error in the compilation.
Here there is an Error, highlighted in color.
There is also a suggestion for correction. std::cout



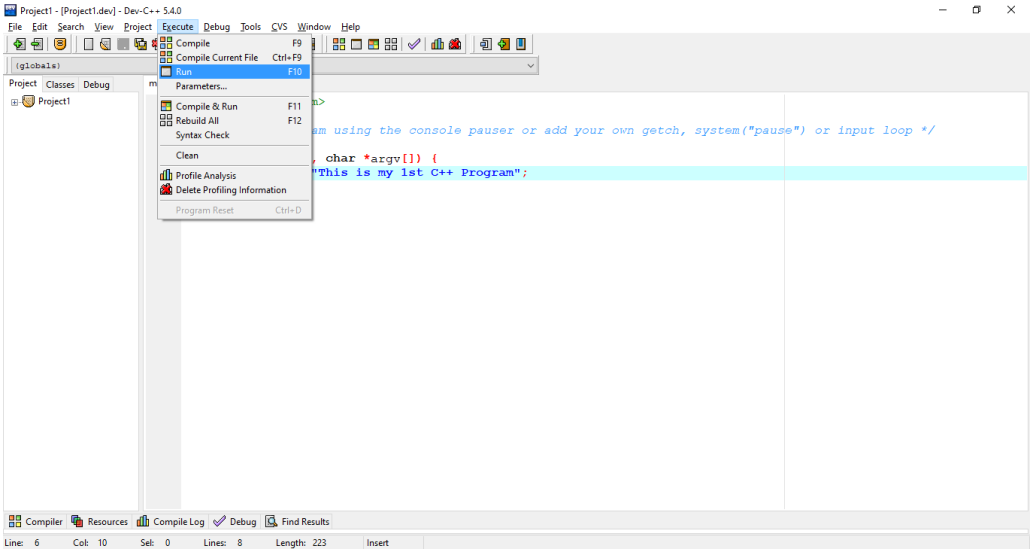

**#11)** Make the necessary correction and Re-compile.



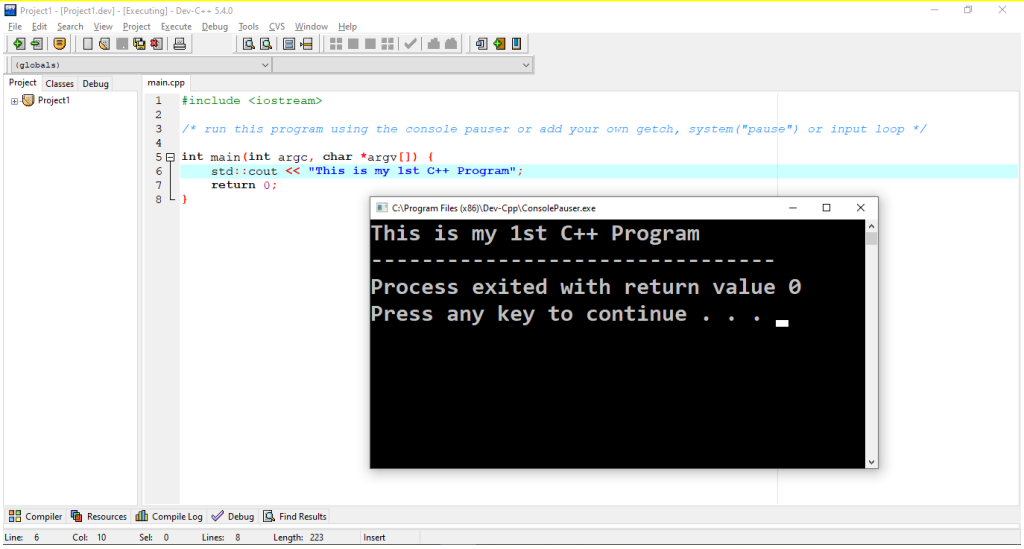

**#12)** Check for any compilation Error.
If OK, Click "Close".




**#13)** Click Main Menu, Execute > Run.

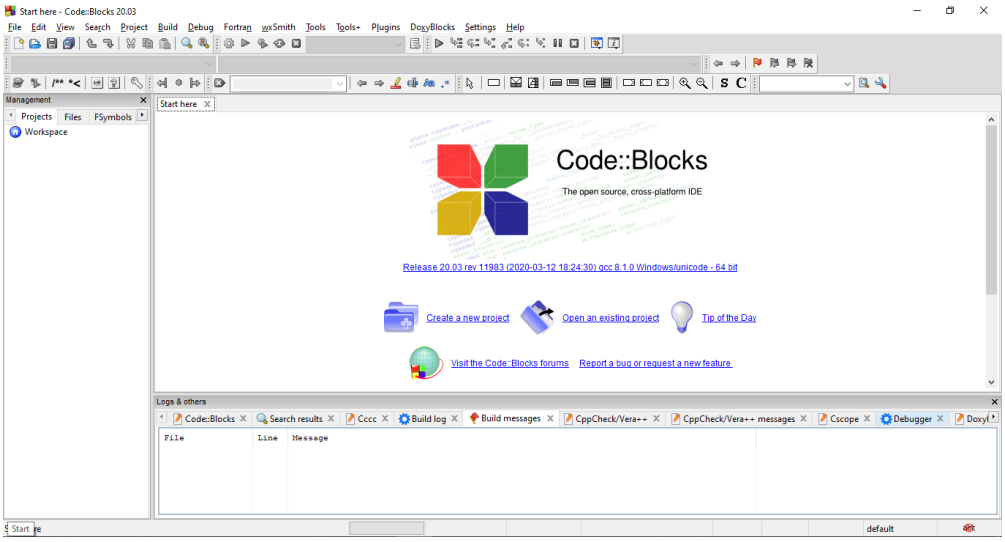



**#14)** Display the Output Result.

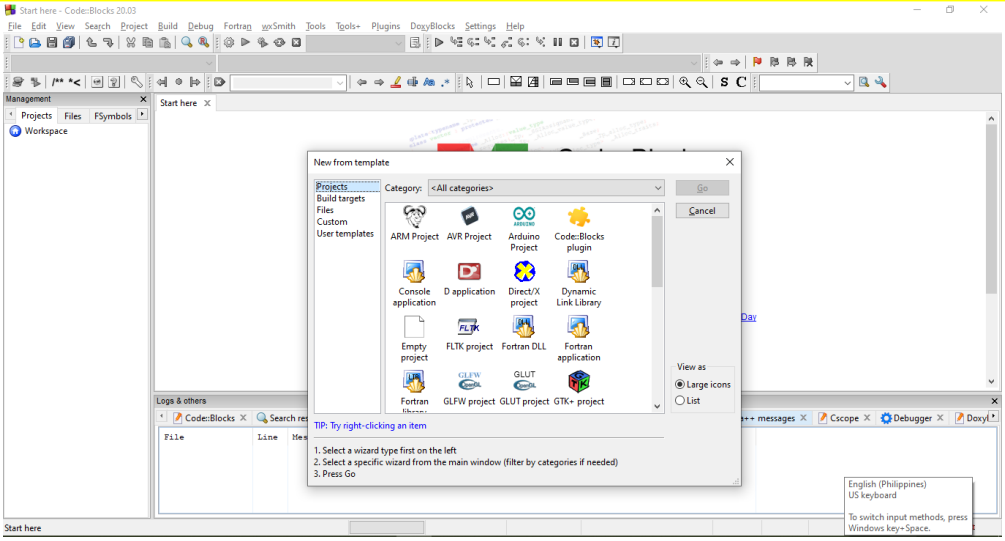



1.2.4 Using Code::Blocks

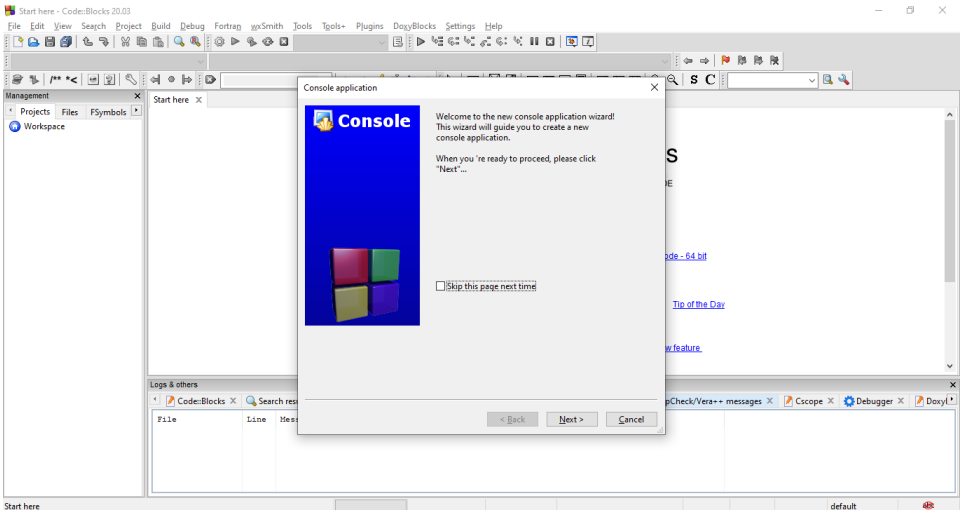
**#1)** Open the Code:Blocks IDE



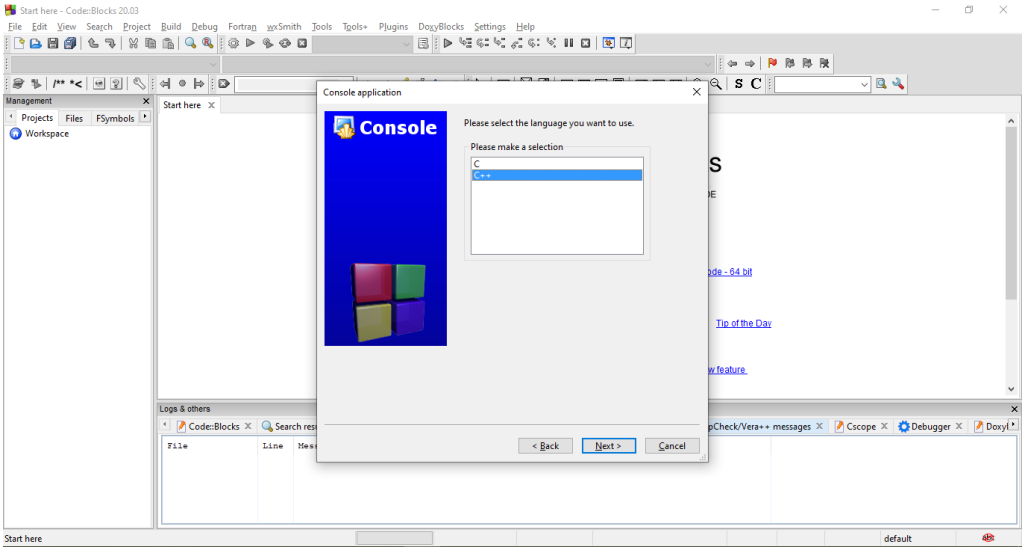

**#2)** Click "Create a New Project"



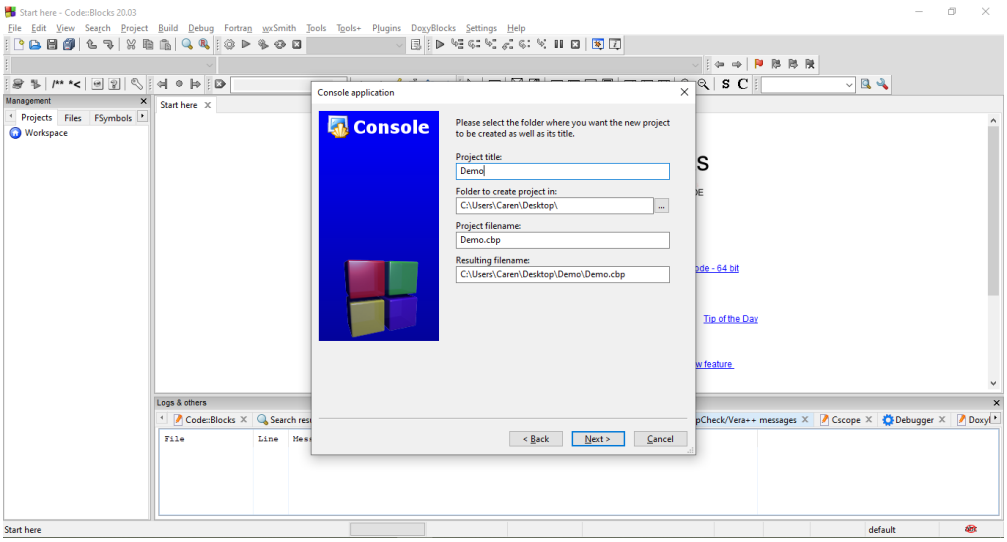

**#3)** Click "Console Application"



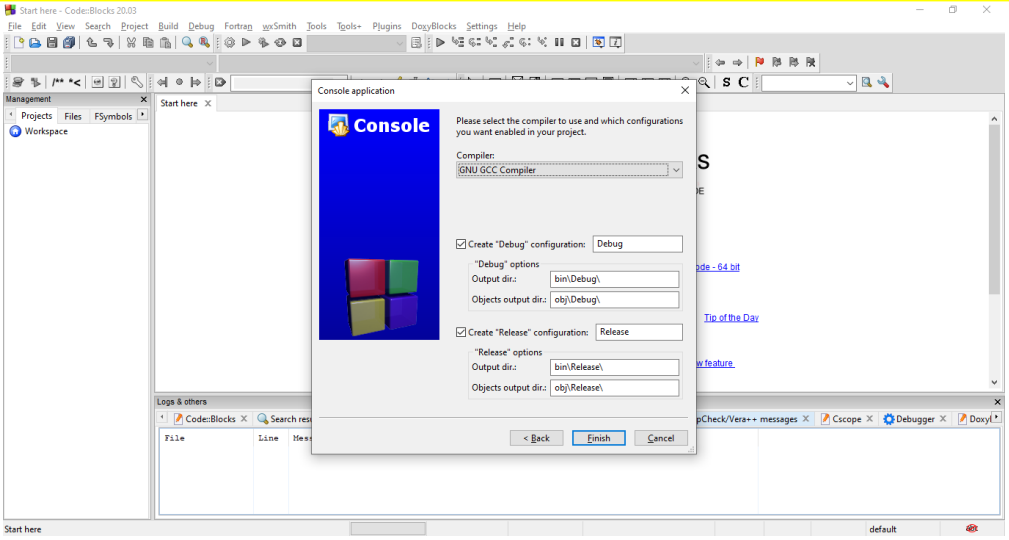

**#4)** Click "Next"



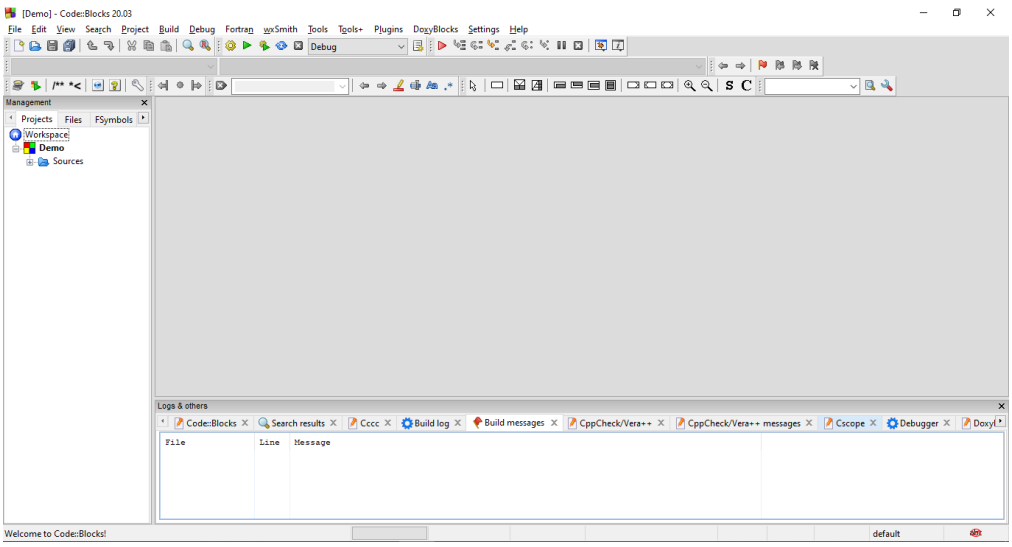

**#5)** Select C++ and Click "Next"



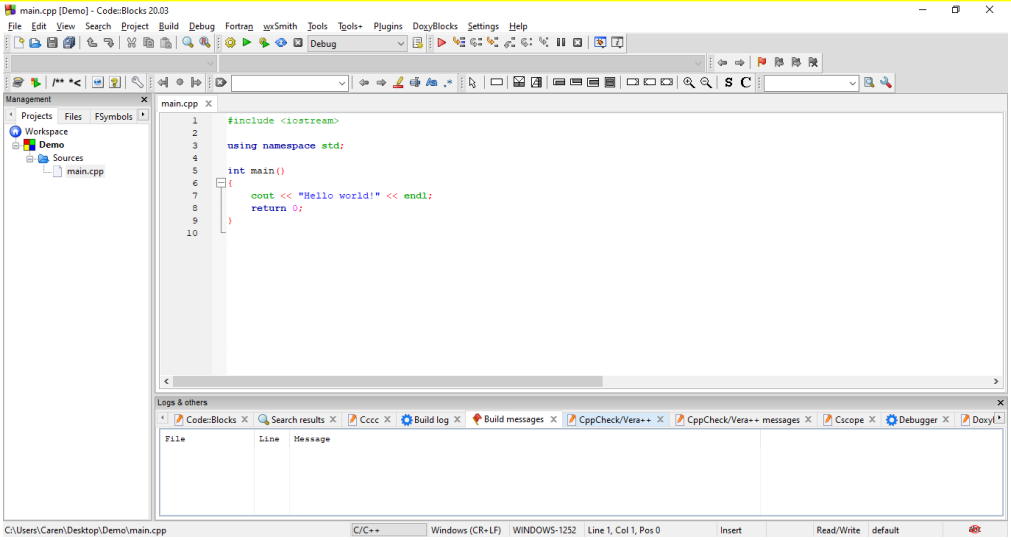

**#6)** Enter "Project Title" and Click "Next"



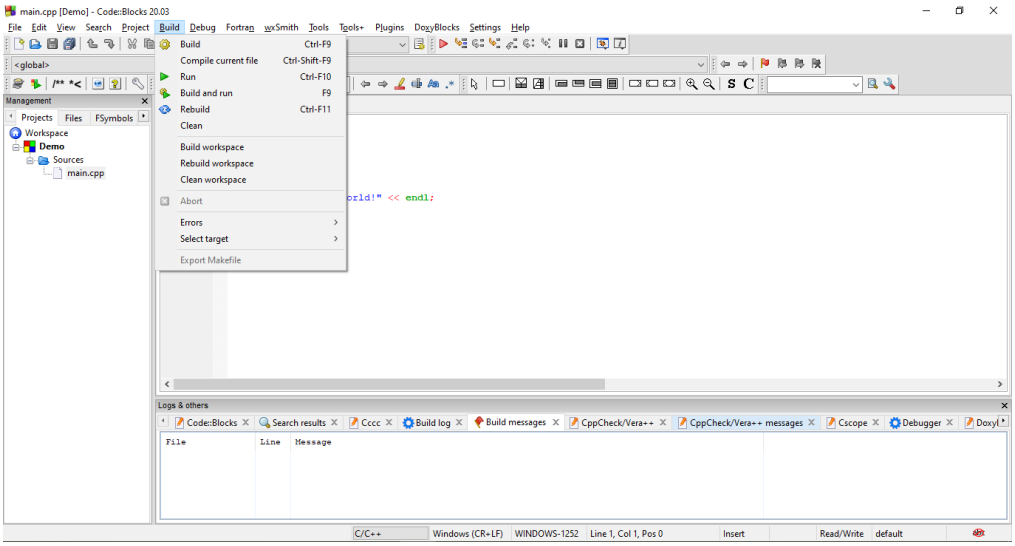

**#7)** Click "Finish"

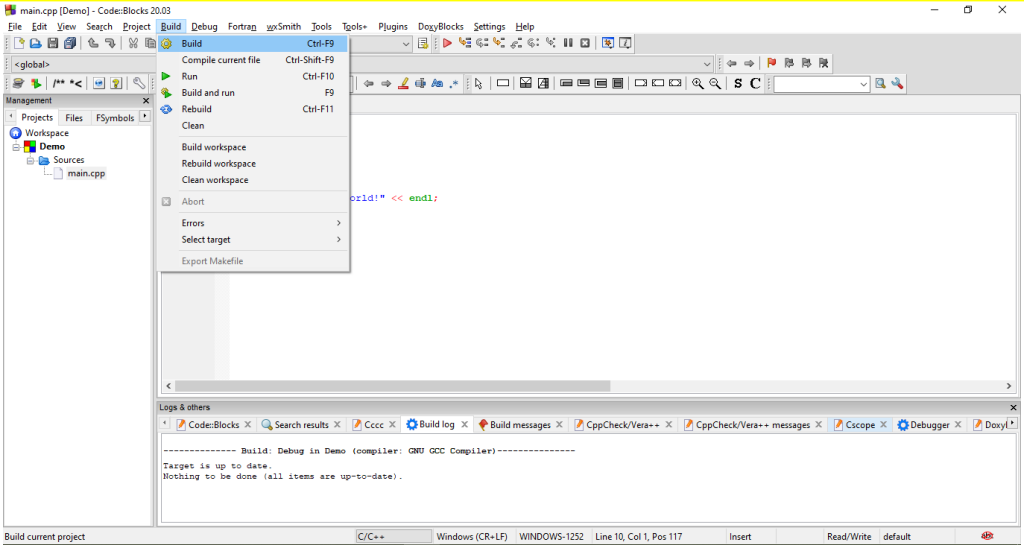



**#8)** Expand "Sources" - Click +



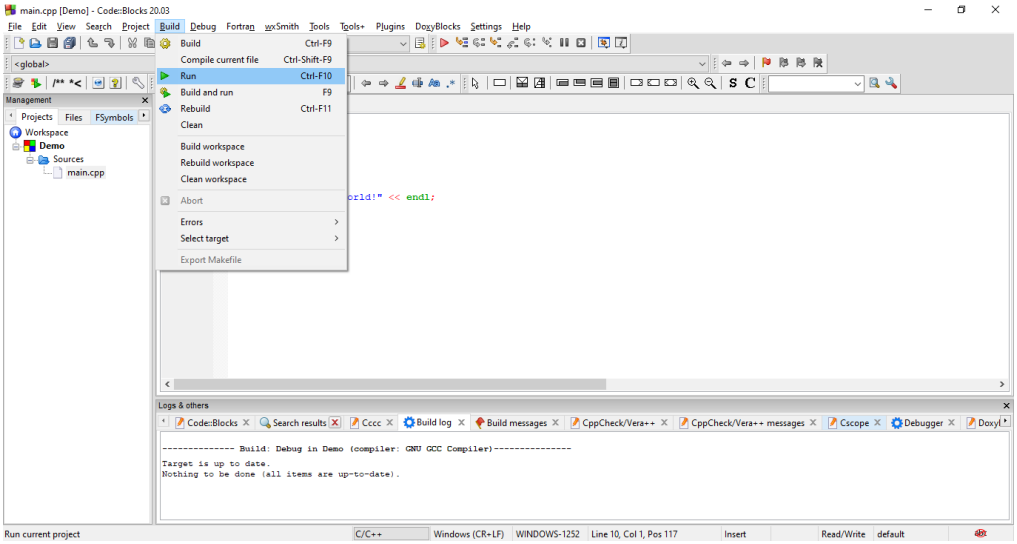

**#9)** From Main Menu, Click Build > Build



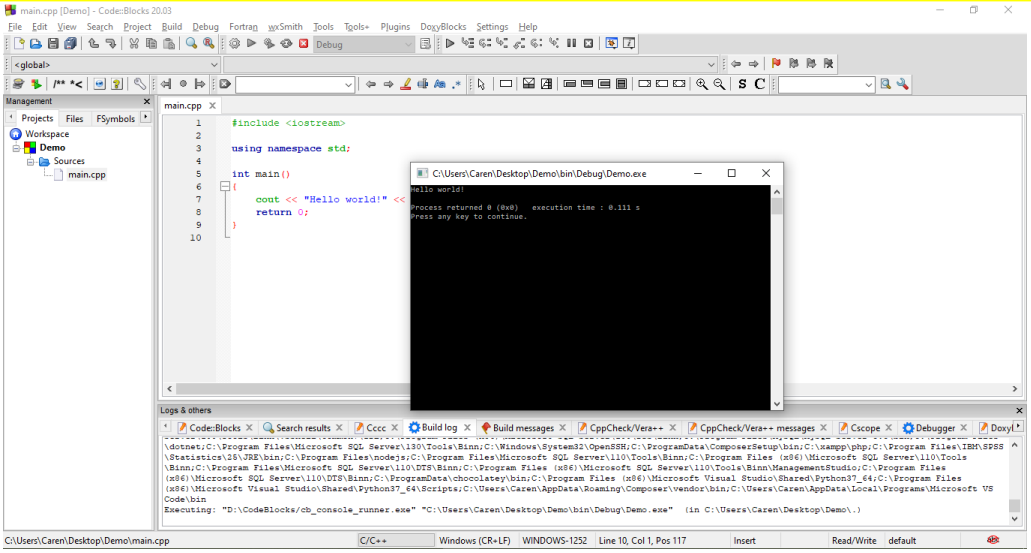



**Check the "Build Log" for any error in the Program Compilation or Build Process. This must be corrected to be able to "Execute or Run" your program.**

**#10)** From Main Menu, Click Build > Run




**#11)** It will Display the OUTPUT Result of the RUN-Program




From the OUTPUT - it shows the following:

1) The Program Name executed with its complete path.

C:\Users\Caren\Desktop\Demo\bin\Debug\Demo.exe

2) The output result

Hello world!

3) The Execution Time.

0.111 seconds

Comprehensive discussions on using Dev-C++ is found at https://www.softwaretestinghelp.com/dev-cpp-ide/, and on using Code::Blocks at https://www.dummies.com/programming/c/basics-of-codeblocks-ide-for-c-programming/

Both Dev-C++ and Code::Blocks are considered two of the best C++ IDEs and source code editor. One difference, however, of the two is that Dev-C++ can be used only on Windows. It does not support other OSs such as Linux and OS X. Code::Blocks, on the other hand, can be used on various platforms such as Windows, Mac OS X, and Linux.

## LEARNING ACTIVITY 2

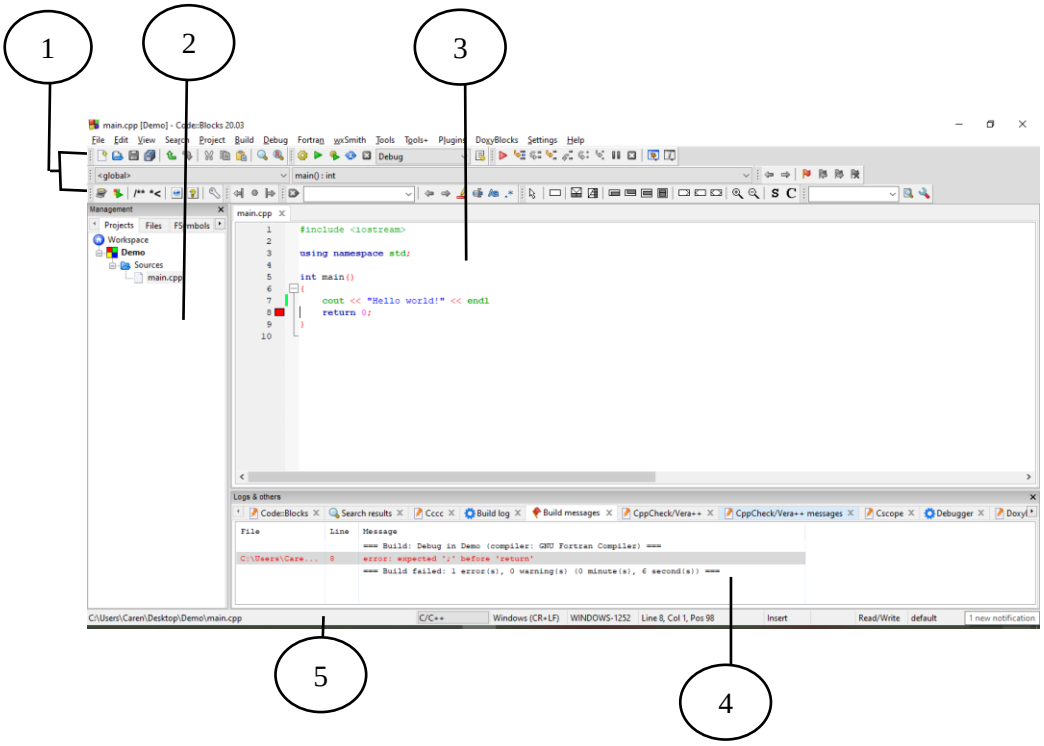
Directions: Test your present knowledge about C++ programming tools by selecting the option that corresponds to the correct word or phrase that will complete the statement. Encircle the letter of the correct answer.

1. Dev-C++ and Code::Blocks are C++ _____ used for developing applications.
   **A.** language **B.** interpreter
   **C.** program **D.** Integrated Development Environment
2. When you run Code::Blocks, you are prompted with _____ window.
   **A.** compilers auto-detection **B.** End User License Agreement
   **C.** language selection **D.** Save Project
3. In C++ IDEs, the _____ is usually in the center right area of the screen where you type code.
   **A.** toolbars **B.** logs **C.** management **D.** editor
4. To build a Dev-C++ project, click Execute->_____ (or click F9).
   **A.** Compile **B.** Run **C.** Build **D.** Debug

5. To execute a C++ program in Code::Blocks, click Build-> _____.
   **A.** Build **B.** Run **C.** Compile **D.** Debug

**Activity**

A. Directions: Identify the numbered main areas in the workspace of Code::Blocks.
Write your answers in the spaces provided.




1._____ 4. _____

2. _____ 5. _____

3. _____

B. Directions: Match your answer in A with the description of the parts below. Write the number and the name of the workspace area described in the spaces provided.

_____A. The bottom of the screen features a window with many, many tabs. Each tab displays information about your programming projects.

_____B. At the bottom of the screen, you see information about the project and editor and about other activities that take place in Code::Blocks.

_____C. The window on the left side of the workspace features four tabs, though you may not see all four at one time. The window provides a handy oversight of your programming endeavours.

_____D. The big window in the center-right area of the screen is where you type code.

_____E. Contains various command buttons, cling to the top of the Code::Blocks window.

## LEARNING CONTENTS (Basic Steps in Problem Solving)

### Introduction

Problem solving is the act of defining a problem; analysing the problem; selecting alternatives for a solution; and implementing a solution. In the context of programming, basic steps in problem solving are problem definition, problem analysis and algorithm design. In this unit, you will be familiarized in these basic steps in preparation for program codes writing in C++.

### Unit learning outcomes

By the end of this unit you should be able to:

- Analyse a given problem by identifying input, process and output.
- Familiarize with keywords and symbols used in pseudocode and flowchart.
- Formulate algorithms and represent them using pseudocode or flowchart.

### 1.3 Basic Steps in Problem Solving

❖ Problem solving for programming consists of three steps: problem definition, problem analysis and algorithm design.

**1.** Defining and Analyzing the Problem

A. Stating the Problem

Problem 1:

Write a program to determine the sum of two numbers.

Problem 2:

Write a program that will compute the area of a circle.

B. Definitive Statement of the Problem

Problem 1: Write a program to determine the sum of two numbers.

Give the computer two numbers. It should be able to add the two numbers and give the result.

***Problem 2:***

Write a program that will compute the area of a circle.

*How will you re-state Problem No 2?*

C. Analyzing the Problem

- NATURE. (What is it about)
- SCOPE. (Limitations)

**Example 1**

**Problem 1:**

*Write a program to determine the sum of two numbers.*

***Nature:*** *Mathematical*

***Scope:*** *Limit the scope to numbers that are integers or whole numbers.*

**Example 2**

**Problem 2:**

*Write a program that will compute the area of a circle.*

***Nature:*** *Geometry.*

***Scope:*** *Allow fractions and decimals as acceptable values. No negative values for the radius should be accepted.*

**B.1. Identifying the Output**

**Problem 1:** Write a program to determine the sum of two numbers.

**Output:**

- ✓ [sum]
- ✓ The sum of the two numbers you gave is: [sum]

**B.1. Identifying the Input**

**Problem 1:** Write a program to determine the sum of two numbers.

**Input:**

- ✓ We need two numbers as the input. (keyed in the keyboard)

**B.1. Identifying the Input**

**Problem 2:**

Write a program that will compute the area of a circle.

**What is the Input Required?**

**C. Defining the Process**

**Problem 1:**

Write a program to determine the sum of two numbers.

**Process:**

- ✓ Enter two numbers using the keyboard. The computer must be able to reject fractions and decimal numbers. The computer will then add the two numbers. The result of the addition will be displayed on the computer screen.

**Problem 2:**

Write a program that will compute the area of a circle.

**Process:**

- ✓ The program must be able to accept a number representing the radius. The computer should make it sure that no negative number will be entered. The computer must compute the area of the circle. Result must be displayed on the screen.

**2. Planning the Solution**

**Algorithm**

- A set of rules for solving a problem in a finite number of steps.

**Overview of an Algorithm**

**I.** STEPS IN COOKING SCRAMBLED EGG

- Prepare all necessary items needed (egg, pan, oil etc.)
- Beat the egg

- Add salt
- Heat pan
- Put oil into the pan
- *Pour beaten egg if pan is already hot enough*
- *Cook egg for at least two minutes*
- *Serve Egg*

**Control Structures**

**Structured programming**

- *is a technique that emphasizes the breaking of a program or algorithm into logical sections, or modules following a universal programming standard. (Breaking of programs into smaller modules)*
- Control structures is a step, or a series of steps, that determines what instructions or set of instructions will be done next.

**Three Basic Structures**

- Sequential
- Selection
- Iteration

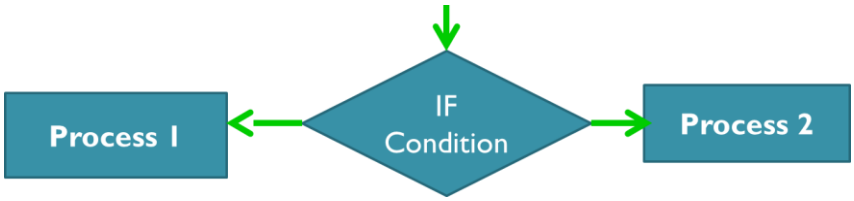
**Sequential**

- is the most straight forward. The program will just follow the instructions one after the other in a sequential manner.

**Example: Taking a Bath**

**Selection**

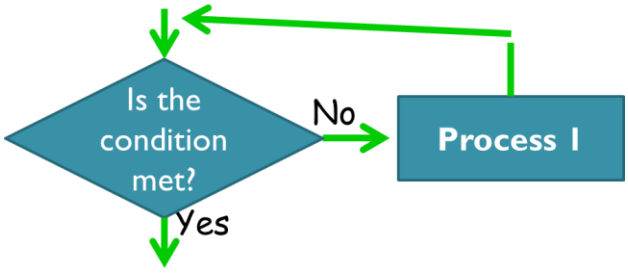
- Let your program or algorithm make a decision. Choose one from two sets of instructions depending on the condition.
  - IF (condition) THEN (Process 1) ELSE (Process 2)




IF (it is dark) THEN (turn on the lights) ELSE (do not turn on the lights)

**Iteration**

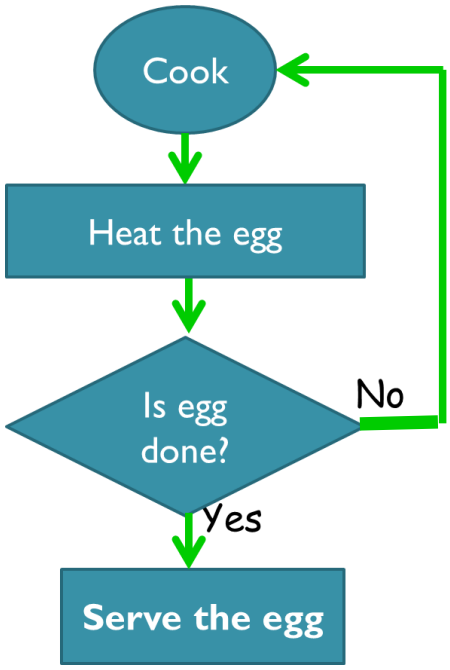
- We use when we want our algorithm or program to repeat a process a specified number of times.
- The structure that repeats an instruction or a set of instructions until a condition is met is known as iteration or loop.




**Whether the scrambled egg is ready to be served or not**

**I.** Beat the egg
**II.** Add salt
**III.** Place the pan in the stove
**IV.** Put on the stove
**V.** Pour cooking oil
**VI.** Heat the oil
**VII.** Put egg
**VIII.** Cook: { here is the example of the control structure}
**IX.** Heat the egg
**X.** IF the egg is done
**XI.** THEN serve the egg
**XII.** ELSE goo to Cook {End pf control structure}

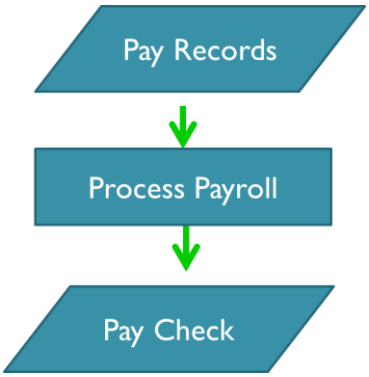
**Iteration control for serving scrambled egg**




**Flowcharting**

- is one method of pictorially representing a procedural (step-by-step) solution to a problem before you actually start to write the computer instructions required to produce the desired results.
- Two Types of Flowchart
  - System (data) flowcharts
    - Defines the major phases of the processing, as well as the various data media used.
  - Programming flowcharts

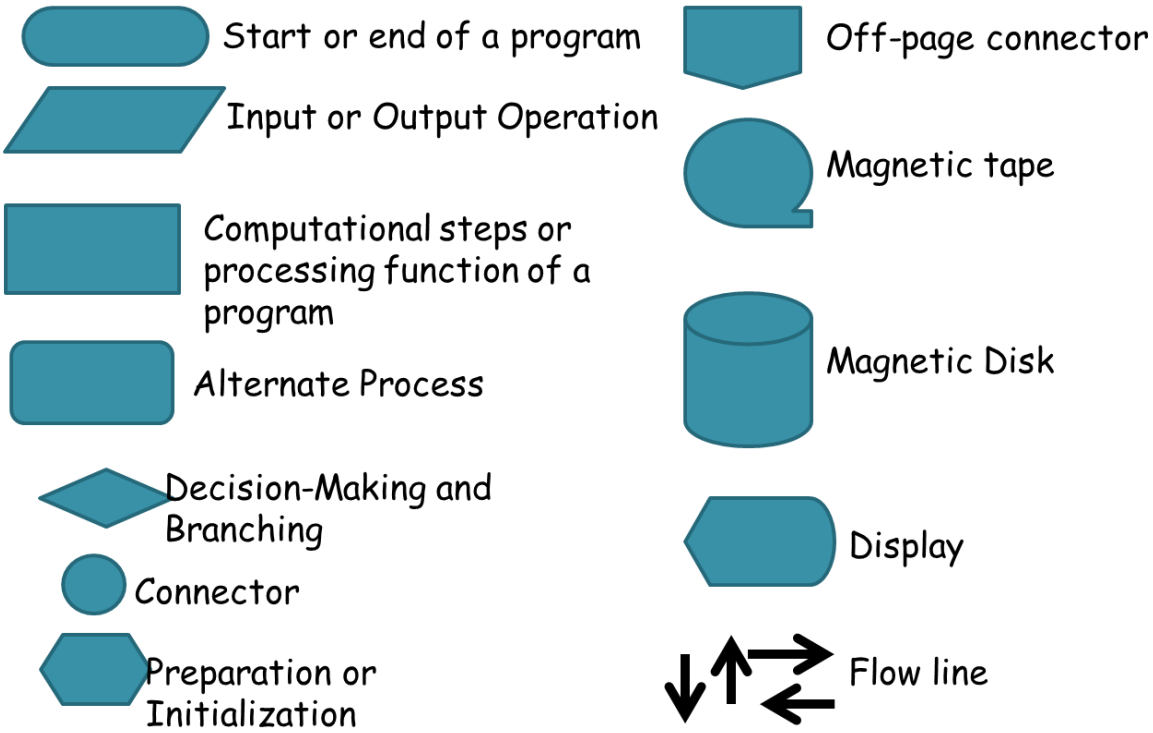
**Example of system flowchart**




**Programming Flowchart**

- Constructed by the programmer to represent the sequence of operations the computer is to perform to solve a specific problem.
- It graphically describes what is to take place in the program.

**Standard Symbols**




**Examples on Flowcharting**

***Problem:*** Make a flowchart to determine the sum of two numbers.

***Definitive Statement:*** Give the computer two numbers. It should be able to add the two numbers and give the result.

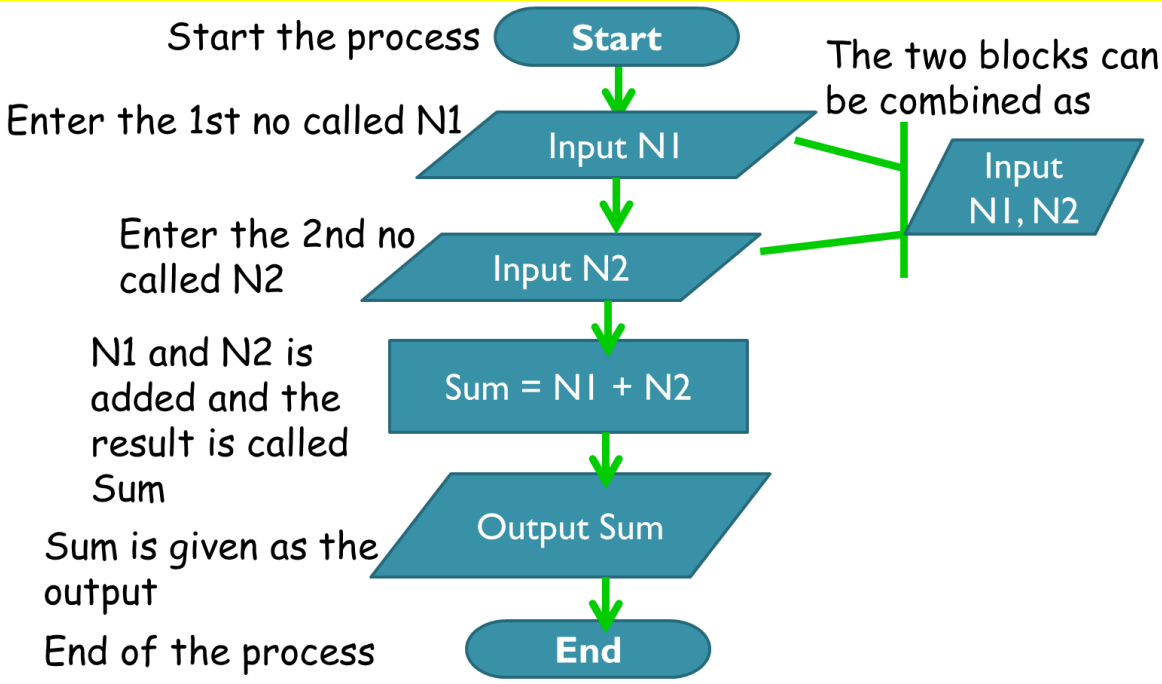
***Problem:*** Make a flowchart to determine the sum of two numbers.

***Definitive Statement:*** Give the computer two numbers. It should be able to add the two numbers and give the result.

***Output:*** [sum] or

The sum of the two numbers is: [sum]

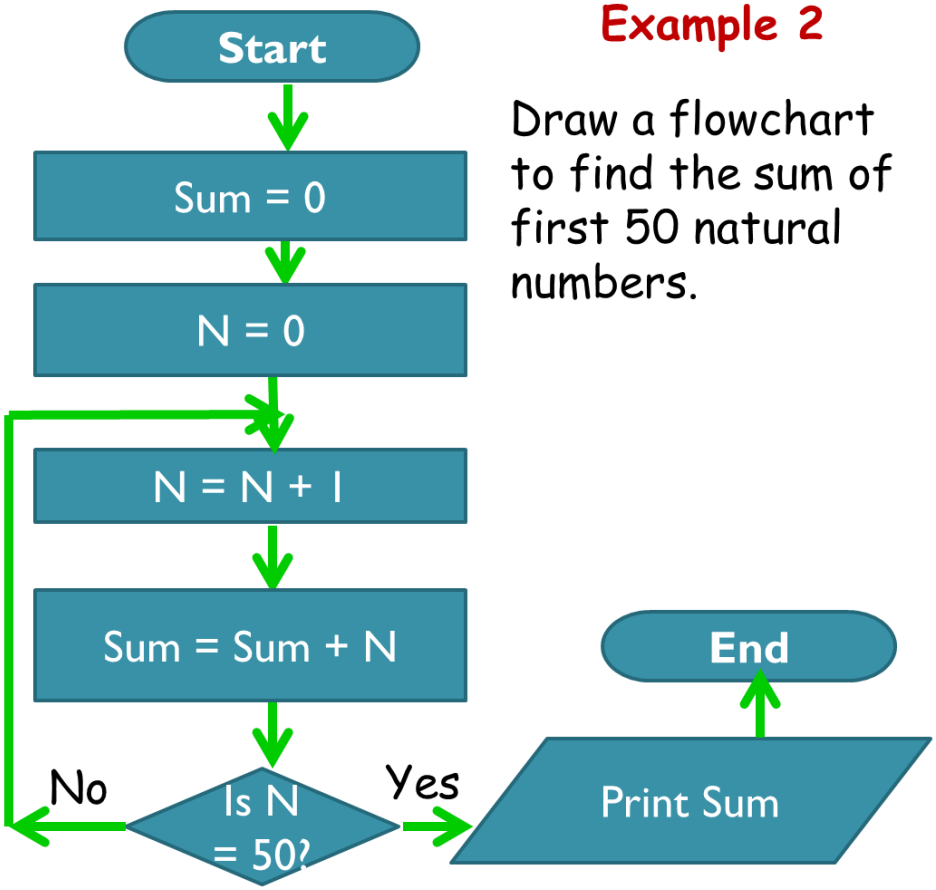
***Input:*** we need two numbers as the input. The program allows both positive and negative integers.

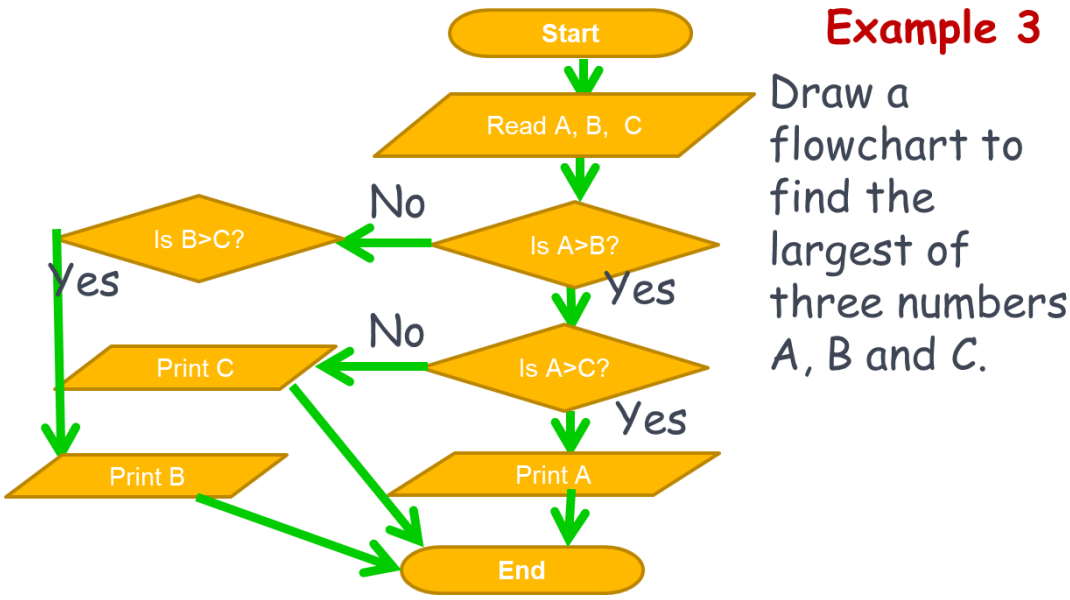



**Example 1**

**Example 2**

Draw a flowchart to find the sum of first 50 natural numbers.






**Pseudocode**

- Uses words and mathematical expressions to represent the logic, or the flow of the program.
- Informal language for writing algorithms.

**Example of Pseudocode**

- ***Problem:*** Write a pseudocode to determine the sum of two numbers.
- ***Definitive Statement:*** Give the computer two numbers. It should be able to add the two numbers and give the result.

***Output:*** The output of the computer should give us a number which is integer or a whole number.

*[sum] or*

*The sum of the two numbers you gave is: [sum]*

| NOTATION | USE | OTHER NOTATIONS USED |
|---|---|---|
| INPUT | Used when reading or entering data in any of the input devices. | READ, ENTER |
| OUTPUT | Used when we wish a value to be displayed. | WRITE, DISPLAY, PRINT |
| BEGIN | Marks the start of a pseudocode. | START |
| END | Marks the end of a pseudocode. | STOP, HALT |
| ← | Storage Operator. Used to indicate that a value is stored in a memory location. | |

**Example of Pseudocode**

***Problem:*** Write a pseudocode to determine the sum of two numbers.

***Definitive Statement:*** Give the computer two numbers. It should be able to add the two numbers and give the result.

***Output:*** The output of the computer should give us a number which is integer or a whole number.

*[sum] or*

*The sum of the two numbers you gave is: [sum]*

***Pseudocode:***

```
BEGIN
  INPUT the first number
  INPUT the seocnd number
  Add the two numbers
  OUTPUT the sum
  END
```

Further readings on flowchart can be found at https://www.visual-paradigm.com/tutorials/flowchart-tutorial/ and on pseudocode at https://blog.usejournal.com/how-to-write-pseudocode-a-beginners-guide-29956242698

## LEARNING ACTIVITY 3

**Self-test:**

Directions: Test your present knowledge about basic steps in problem solving by selecting the option that corresponds to the correct word or phrase that will complete the statement. Encircle the letter of the correct answer.

1. Given the problem "Write a program to determine the sum of two numbers", the input required is/are _____.

**A.** two numbers **B.** sum of two numbers **C.** addition **D.** Display two numbers

2. Rejecting fractions and decimal numbers for data entered describes the _____ of a problem.

**A.** nature **B.** scope **C.** output **D.** Expression

3. Given the problem "Write a program that will compute the area of a circle", "The program must be able to accept a number representing the radius. The computer should make it sure that no negative number will be entered. The computer must compute the area of the circle. Result must be displayed on the screen.", describes the _____ of a program.

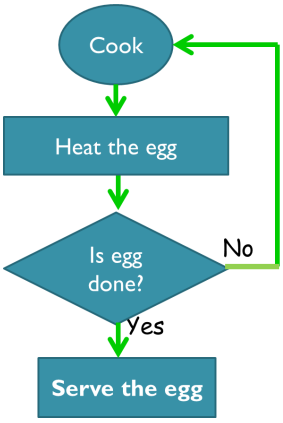
**A.** input **B.** output **C.** process **D.** nature

4. Given the figure below, _____ will take place when "Is it dark?" is True.

**A.** Turn on the lights **B.** Do not turn on the lights

**C.** Go back to "Is it dark?" **D.** End

5. Given the figure below, _____ will repetitively take place if "Is egg done?" is No.




**A.** cook **B.** heat the egg **C.** Is egg done? **D.** serve the egg

6. The statement "OUTPUT the sum" is used to _____.

**A.** enter sum in input device **B.** display sum **C.** store sum **D.** end pseudocode

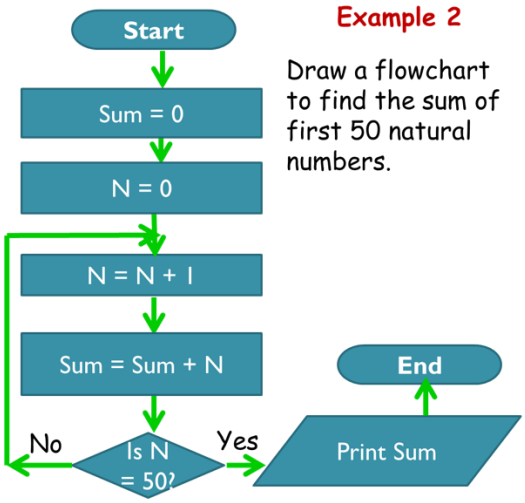
7. The statement "*Sum ← A + B*" _____.

**A.** store Sum in A+B **B.** store A+B in Sum **C.** store A in B **D.** store B in A

8. In the statement *IF (A>B) THEN (Output Sum),* Output Sum will take place when_____.

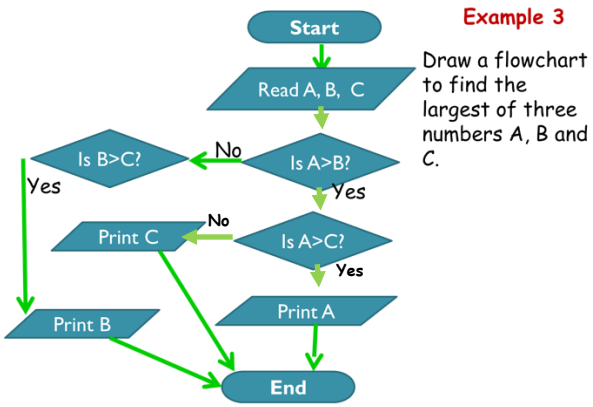
**A.** A>B is yes **B.** A>B is no **C.** A<B is yes **D.** A>=B is no

9. In the flowchart below, Print Sum take place when _____.




**A.** N is not equal to 50 **B.** N is above 50 **C.** N is below 50 **D.** N equals 50

10. In the flowchart below, Is B>C? take place when _____.




**A.** A>B is yes **B.** A>C is yes **C.** A>B is no **D.** A>C is no

**Activity:**

A. Directions: Write your pseudocode or draw your flowchart on separate sheet(s) of paper.

1. Write pseudo code that reads two numbers and multiplies them together and print out their product.
2. Write pseudo code that tells a user that the number they entered is not a 5 or a 6.
3. Draw flowchart that performs the following: Ask a user to enter a number. If the number is between 0 and 10, write the word blue. If the number is between 10 and 20, write the word red. if the number is between 20 and 30, write the word green. If it is any other number, write that it is not a correct color option.
4. Write pseudo code that will count all the even numbers up to a user defined stopping point.
5. Draw flowchart that reads in three numbers and writes them all in sorted order.
6. Write pseudo code that will calculate a running sum. A user will enter numbers that will be added to the sum and when a negative number is encountered, stop adding numbers and write out the final result.
7. There are four types of fruits Apples, Oranges, Bananas and Grapes. Each student can pick up two fruits. There are some conditions which have to be used to pick up the fruits. Draw a flow chart which can take the name of first fruit as an input and print the names of second fruit or fruits that can be picked up.
The conditions are:
• If you pick an apple you can pick banana.
• If you pick orange you can pick grapes.
• If you pick grapes you can pick banana.

*Hint: There will be one input box to read the first fruit, three decision boxes and four output boxes in the flowchart.*

B. Directions: Match the elements of a flowchart and their purpose of use in the following: Write your answer on the first column (Match).

| Match | Purpose | Use |
|---|---|---|
| | Calculate total of A, B, C | Loop |
| | Indicate that the problem has been solved. | Start |
| | Find if a number is greater than the other | Process |
| | Read a number and calculate the factorial of the number | Input |
| | Read three numbers | Stop |
| | Print the total | Decision |
| | Indicate beginning of a problem solving flow | Output |

## LEARNING CONTENTS (Coding and Debugging)

### Introduction

Now that you have learned the basic steps in problem solving and creating algorithms using pseudocode or flowchart, you are now ready to transform your algorithm into C++ program codes. This process is called **coding**. After learning how to use Integrated Development Environment (IDE), you are now ready to write and run your C++ program using either Dev-C++ or Code::Blocks. As soon as you start coding, it is normal for beginners to encounter instances wherein your program would not execute. When this happens, error messages are displayed. The only way you can execute your program is to find the errors or otherwise termed bugs and fix them. We call this process as **debugging** – which means finding the bugs and correcting them.

### Unit learning outcomes

By the end of this unit you should be able to:

- Transform an algorithm into C++ program codes.
- Use IDEs in running and debugging programs.

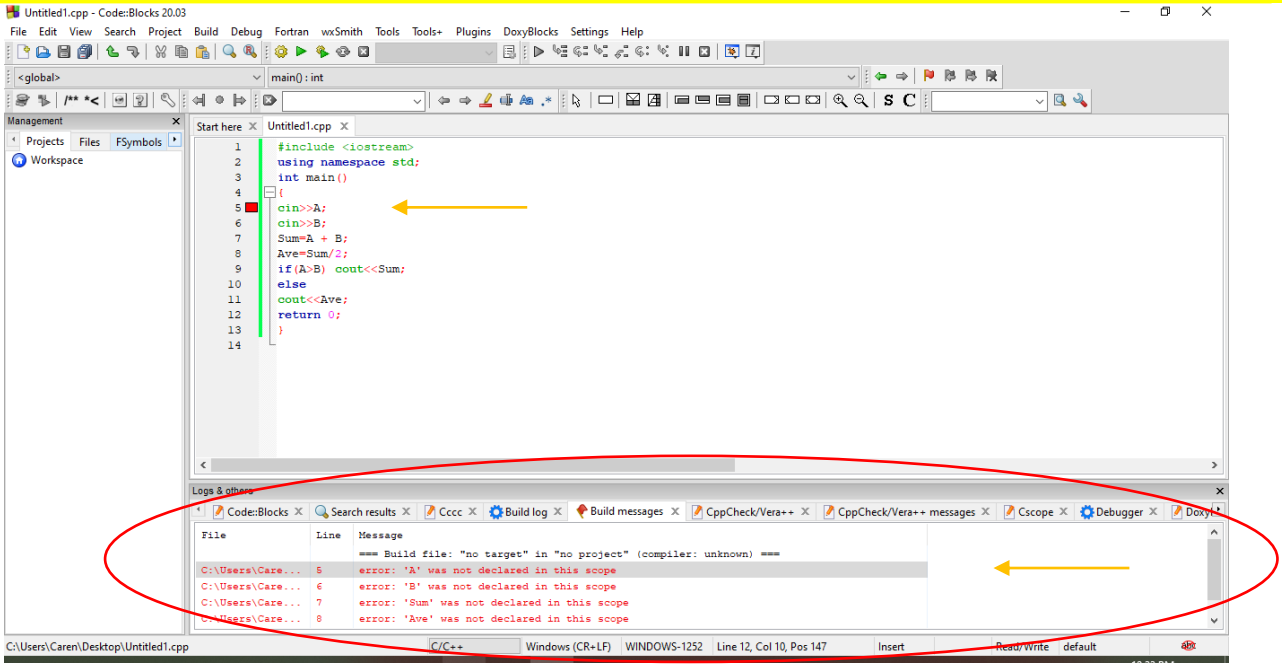
### 1.4 Coding and Debugging

The best way to learn programming language is by writing programs. Although typically the first program a beginner would write is "Hello World", a good way to start is write your algorithm using any representation such as pseudocode or flowchart and then later transform each part into equivalent C++ codes. Table below illustrates lines of a pseudocode translated into equivalent C++ codes.

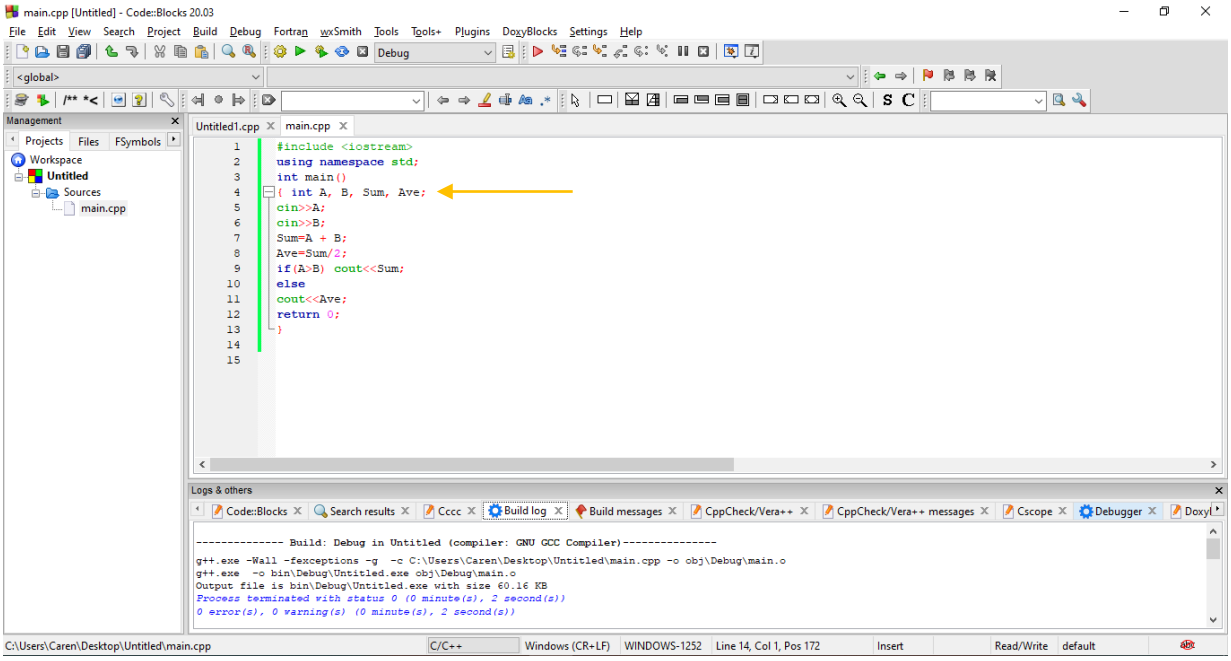
| Pseudocode lines | Equivalent Codes in C++ |
|---|---|
| BEGIN | { |
| INPUT A | cin>>A; |
| INPUT B | cin>>B; |
| Sum← A + B | Sum=A + B; |
| Ave← Sum / 2 | Ave=Sum/2; |
| Output Ave | cout<<Ave; |
| END | } |

However, when you start coding using Code::Blocks or Dev-C++ you need to complete your program with #include directive, enclose the lines of code previously shown, inside void main() or int main() and then add a return 0; just before the close curly brace ( } ). In addition, variables used must be declared before use. If you are not familiar with these terms, do not worry because these will all be discussed to you in detail as you proceed to the other modules.

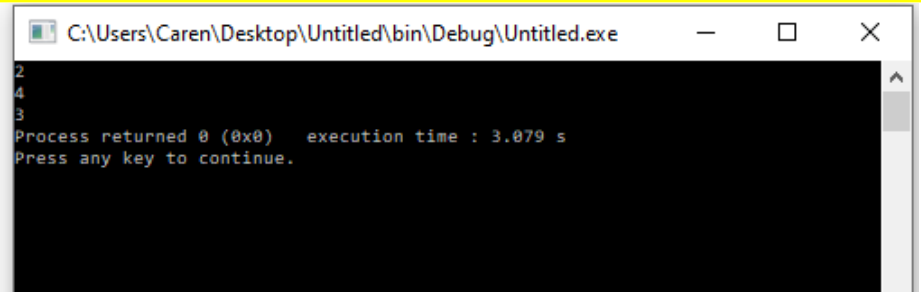
Below is an illustration of how you do coding in C++ using Code::Blocks. Once you have written your code, you can try running your program by going to Build and selecting Build and run. If all is well, an output window displaying results will show. Otherwise, build errors will be displayed.




Observe the encircled part of the window, these are called build messages. If you see error messages displayed here your program will not run until you correct the errors. One of the typical errors that beginners encounter is failure to declare variables, which is the case in this example. Notice that a little red box appear right within a line number indicating that this is the part containing the error highlighted in the error messages pane (see the orange arrow in line 5). Similarly, when you click to the next error messages, the little red box will show you the lines causing the error. To correct all the errors, otherwise known as bugs, all variables should be declared before use. Below illustrates the declaration for the variables, needed in the program.

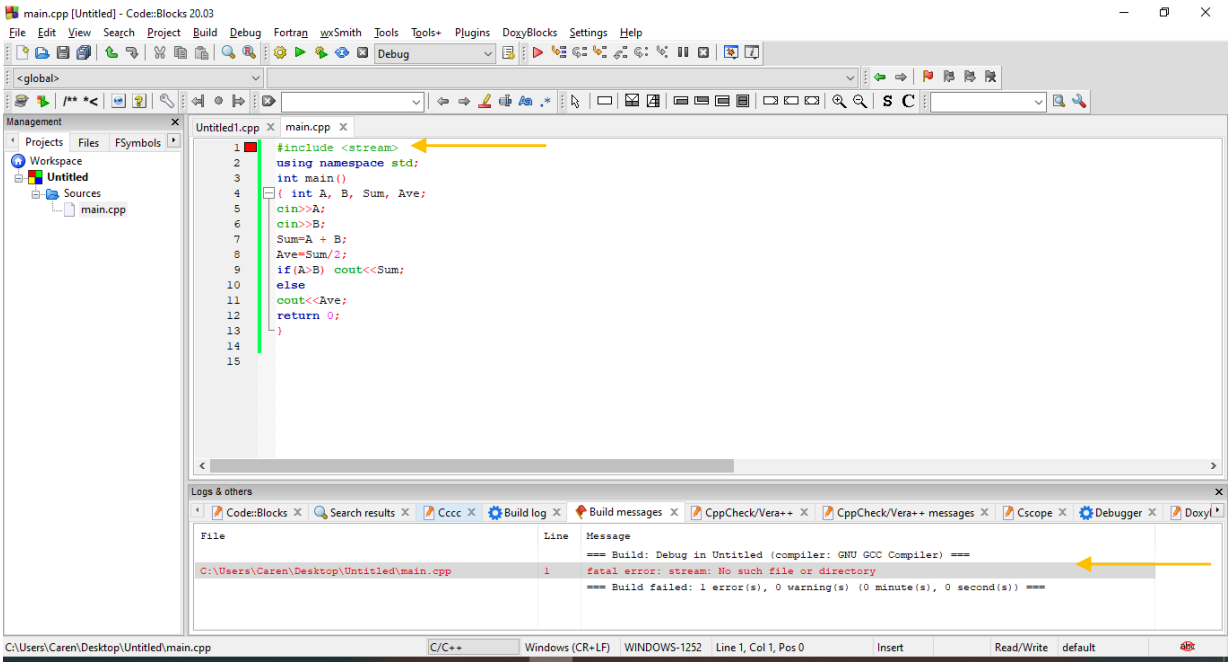



After adding this line (see where orange arrow points), you can try running your program by selecting Build and Run under Build menu. You should see output window as shown. Enter 2 then press enter. Enter 4 then press enter.

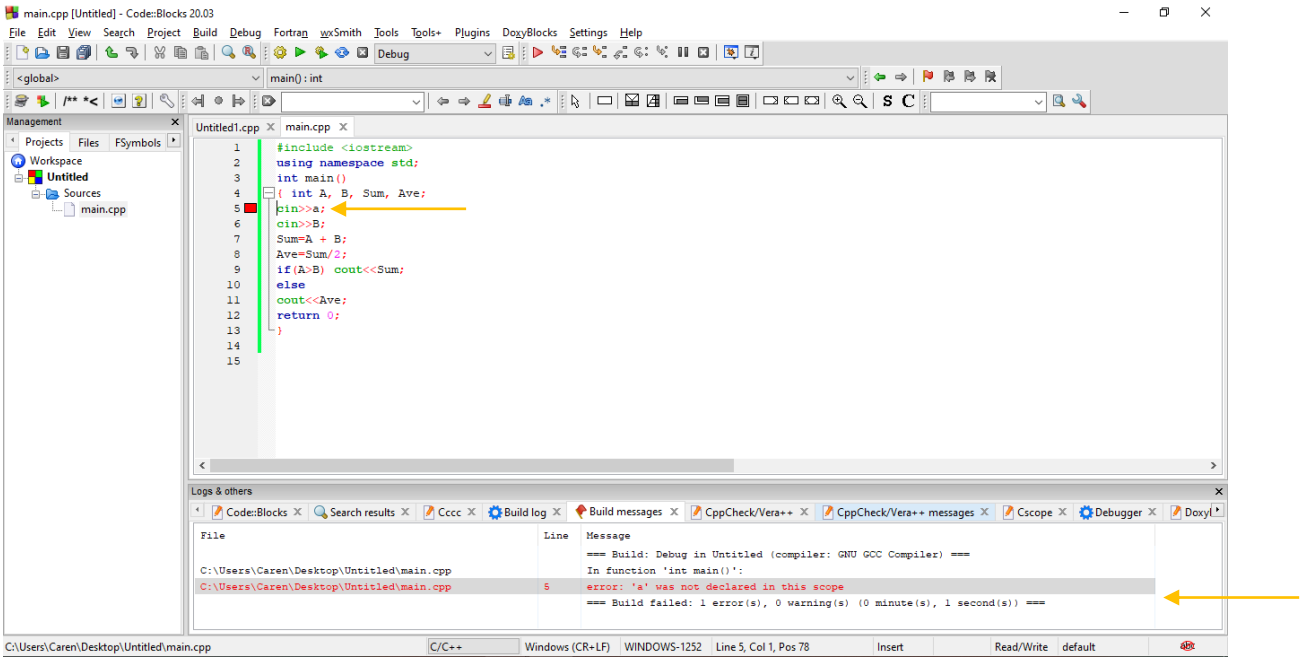



Observe that it displays 3 which is the average of 2 and 4, the numbers you entered previously.

Next, try changing iostream to stream such that your first line becomes #include<stream>. Run the program again. Observe the following error appears.




It is a must that libraries such as iostream be spelled correctly and should be in lowercase letters. Otherwise, the compiler will not be able to locate them from the directory. After trying so, you can put it back to its correct name iostream. C++ is also case sensitive which means lowercase letters are treated differently from uppercase letters. This applies to both keywords such as cin, cout, etc. and identifiers such as A, B, etc. To observe this, try changing line 5 cin>>A; with cin>>a; so your code looks like below:




Notice that when you run the program the following error appears (see orange arrow in error message pane). This is because you have declared variable A yet you have used variable a in your line 5. To fix this error, you will have to put it back to uppercase A in line 5. Alternatively, you can change int A in line 4 with int a and you will observe the program will now work. All of the examples shown previously are syntax errors. Syntax errors occur when you violate the rules of writing in a programming language. This compiler error indicates something that must be fixed before the code can be compiled. All these errors are detected by compiler and thus are known as compile-time errors. There are other types of errors which you may also encounter as you work on more programs in C++.

Comprehensive discussions on errors in C++ is at https://www.tutorialspoint.com/errors-in-c-cplusplus

A programming language, like C++, has its own rules in writing program codes. These rules are referred to as syntax. Thus, violation to these rules causes what we call syntax error.

## LEARNING ACTIVITY 4

**Self-test:**

***Directions:*** Test your present knowledge about coding and debugging by selecting the option that corresponds to the correct word or phrase that will complete the statement. Encircle the letter of the correct answer.

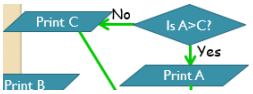
1. INPUT C in pseudocode is _____ when coded in C++.

   **A.** cin>>C; **B.** cin<<C; **C.** cout>>C; **D.** cout<<C;

2. AVE← (A+B)/2 in pseudocode is _____ when coded in C++.

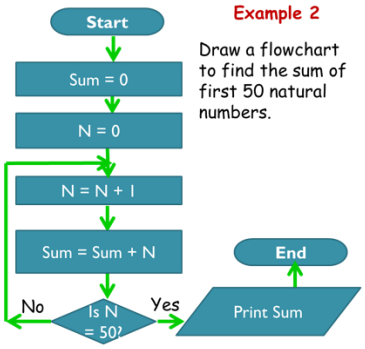
   **A.** AVE(A, B); **B.** AVE=(A+B)/2 **C.** AVE==(A+B)/2; **D.** AVE=(A+B)/2;

3. The flowchart section below is _____ when coded in C++.



**A.** if(A>C) cout<<A; else cout>>C; **B.** if(A>C) cout>>A; else cout>>C;
**C.** if(A>C) cout<<A; else cout<<C; **D.** if(A>C) then cout<<A; else cout>>C;

4. The flowchart below requires a ____ control structures when coded in C++.

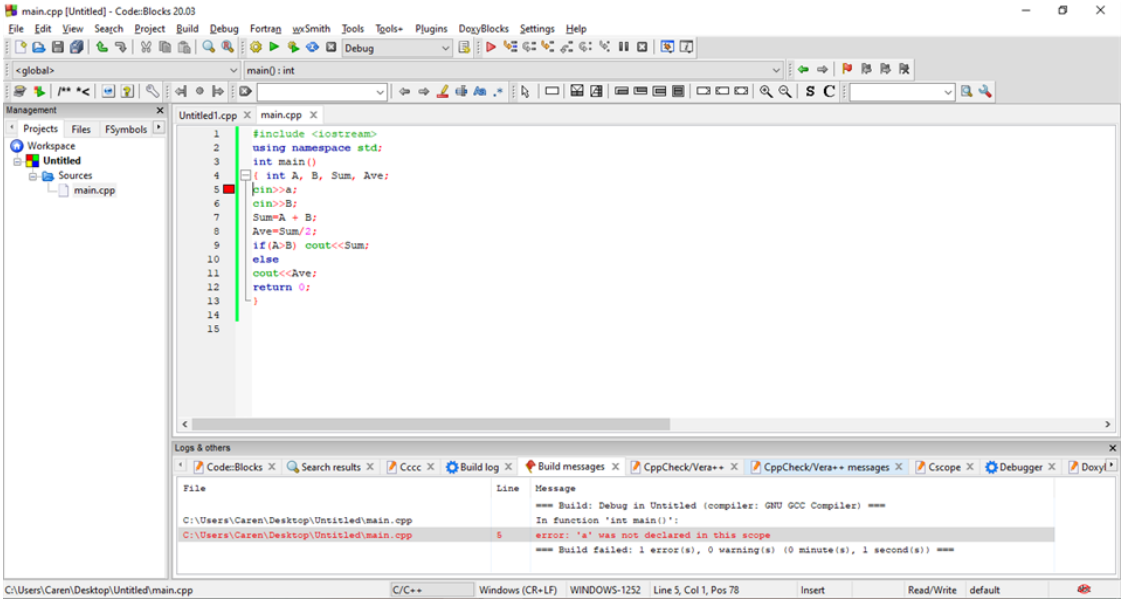



**A.** sequential **B.** selection **C.** iteration **D.** conditional

5. The iteration in the flowchart in #4 is _____ when coded in C++.

**A.** while(N<50){ N=N+1; Sum=Sum+N;} **B.** while(N>50){ N=N+1; Sum=Sum+N;}
**C.** while(N=50){ N=N+1; Sum=Sum+N;} **D.** while(N==50){ N=N+1; Sum=Sum+N;}

6. Consider the error in the program below, line 5 when changed to ____ will make the program run.

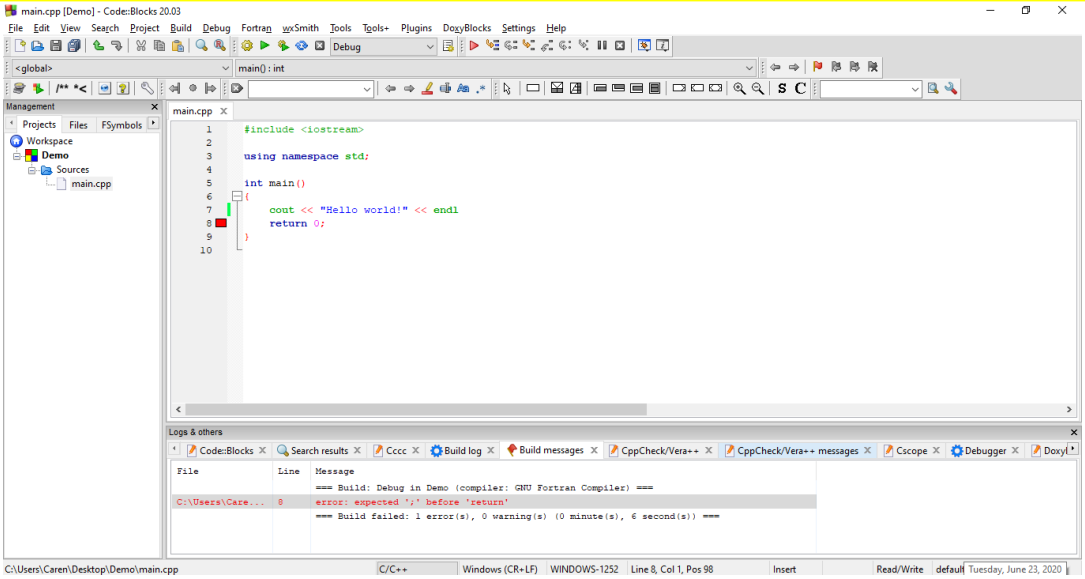



**A.** int a; **B.** cin>>A; **C.** cin<<A; **D.** cout<<A;

7. When line 4 in #6 is changed to ___, the program will run.

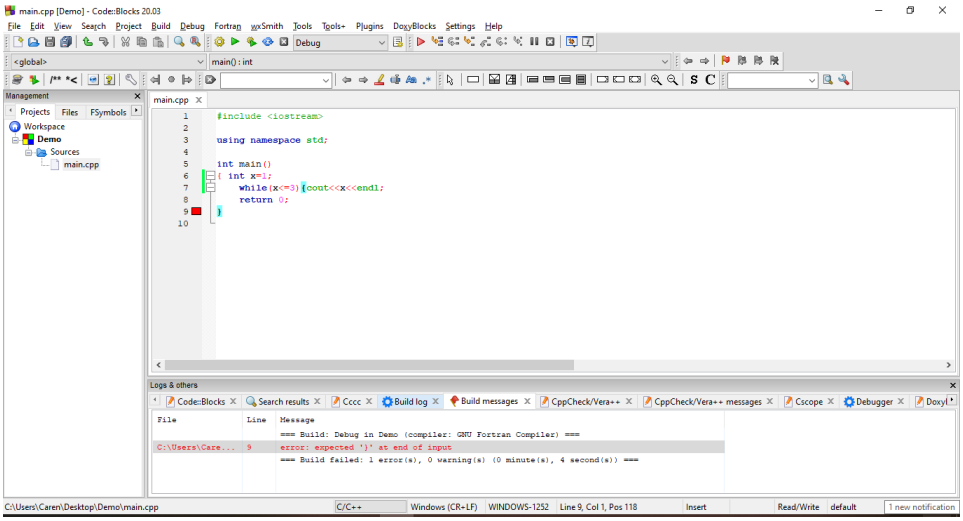
**A.** int a, B, Sum, Ave; **B.** int a; B, Sum, Ave; **C.** int A; **D.** int A

8. To debug the sample program below, Line 7 should end with ___.



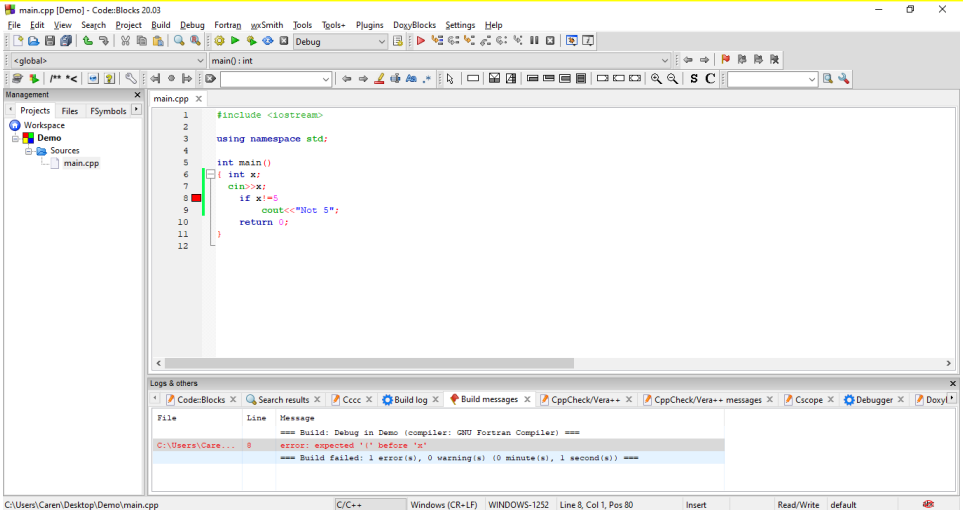

**A.** semicolon (;) **B.** comma (,) **C.** period (.) **D.** curly brace (})

9. In the sample program below, line 7 should end with _____ to debug the program and display numbers 1, 2 and 3.




**A.** semicolon (;) **B.** comma (,) **C.** period (.) **D.** curly brace (})

10. In the sample program below, line 8 should be changed to ____ so that the program will run.




**A.** if x<>5 **B.** if x==5 **C.** if (x!=5) **D.** if {x!=5}

**Activity:**

A. Directions: Write your program codes in a separate sheet of paper.
1. Transform the pseudocode below into C++ codes.

BEGIN
INPUT rate
INPUT hours
GrossPay←rate x hours
Deduction←GrossPay x 0.05
NetPay←GrossPay – Deduction
END

B. Directions: Build and execute the program using Dev-C++ or Code::Blocks

C. Directions: Note the build errors (per line) that you encountered and the changes you made in each line to debug your program. Use the table below.

| C++ Code(s) | Error | Correction |
|---|---|---|
| Example: cin>>rate | Expected ; before return | cin>>rate; |
| | | |

D. Directions: Identify the error in each of the following program.

**1.** #include<iostream>
using namespace std;
void main()
{
int i = 0;
i = i + 1;
cout « i « " ";
/*comment *//i = i + 1;
cout << i;
}

**2.** #include<iostream>
using namespace std;

```
void main()
{
    short i=2500, j=3000;
    cour>> "i+j="> -(i+j);
}
```

**3.**
```
#include<iostream>
using namespace std;
void main()
{
    int i=10, j=5;
    int modResult=0;
    int divResult=0;
    modResult = i%j;
    cout<<modResult<<" ";
    divResult = i/modResult;
    cout<<divResult;
}
```

**4.** Find errors, if any, in the following C++ statements.
**(a)** cout.<<“x=” x;
**(b)** m = 5; // n = 10; // = m + n:
**(c)** cin >>x; >>y;
**(d)** cout <<\h ‘Name:” <<name;
**(e)** cout <<“Enter value:”; cin >> x:
**(f)** /*Addition*/ z = x + y;

Use the table below.

| Question | Correction |
|---|---|
| Example: a | cout<<a; |
| | |

## SUMMARY

- Computer systems consist of hardware and software components. The computer hardware components are physical parts found within computer case while some parts may be connected externally. The computer system needs software - which directs the computer hardware.
- Computer systems have five main components namely input and output devices, processor, main memory and secondary memory.
- High level languages resemble human languages and are designed to be easy to read and write.
- Low level languages pertain to assembly language that needs to be translated to machine language (zeros and ones).
- Programming languages are the foundation for developing applications.
- Machine Language (first generation) is the computer’s ‘native language’ and is composed of binary digits(0s, 1s) - the only language that computers understand. Assembly Language (second generation) has one-to-one correspondence to machine language (mnemonic rather that binary digits). Procedural Languages (third generation) is closer to natural language; uses common words rather than abbreviated mnemonics. Examples include C, C++, Java, Fortran and QuickBasic. Non-procedural Languages (fourth generation) allows the user to specify the desired result without having to specify the detailed procedures needed for achieving the result. Natural Language Programming Languages (fifth generation (intelligent) languages) translates natural languages into a structured, machine-readable form.

- Current Programming Languages include Object-Oriented Programming Languages (OOP), Visual Programming Languages, Hypertext Markup Language (HTML), Extensible Markup Language (XML) and Virtual Reality Modeling Language (VRML).
- Compilers translate high-level language to machine language.
- Linkers combine the object code for the programs we write and the object code for the pre-compiled routines into the machine language program the CPU can run.
- The compiler reads the entire program and converts it into object code, which is a translation of the program source code into a form that the computer can execute directly. Free IDEs available are Bloodshed Dev-C++ and Code::Blocks.
- To have an IDE, download first installer for Bloodshed Dev-C++ and Code::Blocks.
- Install by running the program installer and hitting next keys until installation wizard says it is finished.
- After installation, you can begin with your programming tasks by opening the IDE. Then creating a new project or new console application.
- Use the code editor to begin writing your program.
- Problem solving for programming consists of three steps: problem definition, problem analysis and algorithm design.
- Problem definition is the step wherein what the user wants is identified by writing a definitive statement of the problem.
- Problem analysis involves analyzing what the problem is about (nature) and its limitations (scope). Input and output are identified and the process is defined. Input pertains to data that is assigned or entered by user while output is the expected results that is to be returned. Algorithm design consists of formulating a set of rules for solving a problem (referred to as algorithm) and using a representation technique such as pseudocode or flowchart.
- Structured programming is a technique that emphasizes the breaking of a program or algorithm into logical sections, or modules following a universal programming standard.
- Control structures is a step, or a series of steps, that determines what instructions or set of instructions will be done next.
- The three basic control structures are sequential, selection and iteration.
- Sequential is the most straight-forward. The program will just follow the instructions one after the other in a sequential manner.
- Selection let your program or algorithm make a decision. Choose one from two sets of instructions depending on the **condition**.
- Iteration is used when we want our algorithm or program to repeat a process a specified number of times.
- The structure that repeats an instruction or a set of instructions until a condition is met is known as iteration or **loop**.
- Flowcharting is one method of pictorially representing a procedural (step-by-step) solution to a problem before you actually start to write the computer instructions required to produce the desired results.
- Programming flowchart is constructed by the programmer to represent the sequence of operations the computer is to perform to solve a specific problem.
- Pseudocode uses words and mathematical expressions to represent the logic, or the flow of the program.
- To run program, from Main menu click Build>Run in Code::Blocks and Execute>Compile in Dev-C++.
- Output result can be seen in the output window.
- Writing a program using a language is referred to as coding.
- A good way to start coding in C++ is write your algorithm using any representation such as pseudocode or flowchart and then later transform each part into equivalent C++ codes.

- To write codes and execute programs, you can use IDEs such as Dev-C++ or Code::Blocks. After writing your codes, you can run the program by going to Build menu, then selecting Build and run. The output window should display if your program is free of errors.
- When your program has errors in it, it will never run but build error message(s) will appear on the build messages log. When you click the error message, you will be pointed to the line causing the error. You can now fix the error and execute the program. The process of finding and correcting errors otherwise known as bugs is called debugging.

## REFERENCES

1. Zak, D. (2016). An Introduction to Programming with C++ (8E), pages 1-14
2. Davis, Stephen R. C++ For Dummies 7th edition. John Wiley & Sons, Inc., Hoboken, New Jersey, 2014
3. Bronson, Gary J. C++ Programming: Principles and Practices for Scientists and Engineers. Cengage Learning, 2013.
4. Scholl, T., Nugent, G. C++ Programming Problem Analysis to Program Design. Cengage Learning, 2011

**Online Reading Materials:**

- http://cplusplus.com/doc/tutorial/introduction/
- https://www.learncpp.com/cpp-tutorial/introduction-to-programming-languages/
- https://www.learncpp.com/cpp-tutorial/introduction-to-cpp-development/
- https://www.learncpp.com/cpp-tutorial/introduction-to-the-compiler-linker-and-libraries/
- https://www.learncpp.com/cpp-tutorial/installing-an-integrated-development-environment-ide/
- https://beginnersbook.com/2017/08/c-plus-plus-tutorial-for-beginners/
- https://www.tutorialspoint.com/cplusplus/index.html